



Nokia Presence Server 1.1

Service Developer's Guide

Reference Document

The information in this document is subject to change without notice and describes only the product defined in the introduction of this documentation. This document is intended for the use of Nokia's customers only for the purposes of the agreement under which the document is submitted, and no part of it may be reproduced or transmitted in any form or means without the prior written permission of Nokia. The document has been prepared to be used by professional and properly trained personnel, and the customer assumes full responsibility when using it. Nokia welcomes customer comments as part of the process of continuous development and improvement of the documentation.

The information or statements given in this document concerning the suitability, capacity, or performance of the mentioned hardware or software products cannot be considered binding but shall be defined in the agreement made between Nokia and the customer. However, Nokia has made all reasonable efforts to ensure that the instructions contained in the document are adequate and free of material errors and omissions. Nokia will, if necessary, explain issues which may not be covered by the document.

Nokia's liability for any errors in the document is limited to the documentary correction of errors. NOKIA WILL NOT BE RESPONSIBLE IN ANY EVENT FOR ERRORS IN THIS DOCUMENT OR FOR ANY DAMAGES, INCIDENTAL OR CONSEQUENTIAL (INCLUDING MONETARY LOSSES), that might arise from the use of this document or the information in it.

This document and the product it describes are considered protected by copyright according to the applicable laws.

NOKIA logo is a registered trademark of Nokia Oyj.

Other product names mentioned in this document may be trademarks of their respective companies, and they are mentioned for identification purposes only.

Copyright © Nokia Oyj 2003. All rights reserved.

Contents

	Summary of changes	5
1	About this document	6
2	Overview	8
2.1	Environment	9
2.2	General Nokia Presence Server usage model	10
2.3	Presence Server Web Service Interface versions.....	12
3	Nokia Presence Server usage scenarios	13
3.1	Login and Presence update	13
3.2	Contact lists and Authorisation.....	14
3.3	Presence query	15
3.4	Presence subscription	15
3.5	User search	16
4	Presence Web Services Interface 1.0	17
4.1	Transactions.....	18
4.2	Data types	19
4.3	Error codes.....	21
4.4	Operations.....	23
4.4.1	Operation #1: GetPresence.....	23
4.4.2	Operation #2: UpdatePresence	24
4.4.3	Operation #3: SubscribePresence	25
4.4.4	Operation #4: UnsubscribePresence	26
4.4.5	Operation #5: PresenceNotification.....	27
4.4.6	Operation #6: GetSubscriptionList	27
4.4.7	Operation #7: GetWatcherList.....	28
4.4.8	Operation #8: CreateContactList.....	29
4.4.9	Operation #9: DeleteContactList	30
4.4.10	Operation #10: GetContactLists	31
4.4.11	Operation #11: ManageContactList.....	32
4.4.12	Operation #12: CreateAttributeList.....	33
4.4.13	Operation #13: DeleteAttributeList	34
4.4.14	Operation #14: GetAttributeList.....	35
4.4.15	Operation #15: Search	36
4.4.16	Operation #16: StopSearch.....	38
4.5	Authentication and authorisation	39
5	Presence Web Services Interface 1.1	40
5.1	Transactions.....	40
5.2	Data types	42
5.3	Error codes.....	44
5.4	Operations.....	46
5.4.1	Operation #1: GetPresence.....	46

5.4.2	Operation #2: UpdatePresence	47
5.4.3	Operation #3: SubscribePresence.....	48
5.4.4	Operation #4: UnsubscribePresence.....	49
5.4.5	Operation #5: PresenceNotification.....	50
5.4.6	Operation #6: GetSubscriptionList.....	50
5.4.7	Operation #7: GetWatcherList	51
5.4.8	Operation #8: CreateContactList	52
5.4.9	Operation #9: DeleteContactList	54
5.4.10	Operation #10: GetContactLists	54
5.4.11	Operation #11: ManageContactList.....	55
5.4.12	Operation #12: CreateAttributeList	57
5.4.13	Operation #13: DeleteAttributeList	58
5.4.14	Operation #14: GetAttributeList	58
5.4.15	Operation #15: Search	60
5.4.16	Operation #16: StopSearch	62
5.5	Authentication and authorisation	63
6	Creating a WSI-enabled service	64
6.1	Required information	64
6.2	Creating a WSI application	65
6.3	Sample WSI application	65
6.4	Receiving presence notifications to WSI applications	70
	Appendix A: Presence attributes	72
	Appendix B: Extended presence information.....	92
	Appendix C: Examples of Presence WSI SOAP messages	93
	References	96
	Glossary	97
	Index	99

Summary of changes

Date	Issue	Summary of changes
22 October 2003	2-0 en	First issue for Nokia Presence Server Release 1.1.
25 August 2003	1-0 en	First issue for Nokia Presence Server Release 1.0.

1

About this document

This guide provides information on the development of presence-enabled services using Nokia Presence Server.

The scope of this guide is to describe the elements of Nokia Presence Server that are relevant in service creation, and to explain how to create presence-enabled network applications and how to take them into use.

This guide includes the following chapters:

- *About this document*
Describes the contents of the document.
- *Overview*
Contains a brief description of the environment in which Presence Server typically operates.
- *Nokia Presence Server usage scenarios*
Presents some possible ways of using Presence Server in different systems.
- *Presence Web Services Interface 1.0*
Describes the Presence Web Services Interface (WSI) 1.0.
- *Presence Web Services Interface 1.1*
Describes the Presence Web Services Interface (WSI) 1.1.
- *Creating a WSI-enabled service*
Describes the creation of WSI-enabled services.
- *Appendix A: Presence attributes*
Lists the currently supported presence attributes.
- *Appendix B: Extended presence information*
Describes how to define manufacturer, vendor or service provider specific extension attributes and extension fields.
- *Appendix C: Examples of Presence WSI SOAP messages*
Contains two examples of SOAP requests/responses.

The target audience for this guide includes service developers who design and implement presence-enabled services, operators' own service developers, and 3rd

party service developers. This document is for anyone who wants to benefit from the facilities provided by Nokia Presence Server.

The audience is expected to be familiar with developing Web services. This includes the following technologies and mark-up languages:

- Hypertext Transfer Protocol (HTTP)
- Simple Object Access Protocol 1.1 (SOAP) [4]
- Web Service Definition Language 1.1 (WSDL) [5]
- Extensible Mark-up Language (XML)

Other possible technologies may include dynamic content creation with Java servlets.

2 Overview

Nokia Presence Server is a server software product enabling the use of services that utilise presence. Presence is a dynamic (variable) profile of the user, visible to others and used to represent oneself, share information and control services.

Applications can utilise presence information in, for example, corporate directory or scheduling and time management systems. In addition, the Presence Web Services Interface (WSI) can be used for network based or triggered presence updates and monitoring for users or other entities that may have associated presence data.

Below are two examples of typical services that utilise presence:

Radio station: A service application sharing presence information

Nokia Presence Server users may store the contact card of their preferred radio station into their phonebook. The radio station updates the track information and related visual content in the presence server for each song playing.

If users hear a song they like, they can request a presence update for the radio contact card from their phonebook. The terminal client does a one-time fetch from Nokia Presence Server on the radio presence information

The user can see the name of the current song/singer, as well as related information, for example the album cover, under the radio contact in the phonebook.

Corporate directory enhanced by presence

Corporate directory is a phonebook that contains the contact information of the employees of a company. When it is enhanced by presence, an employee's profile is displayed along other information.

Nokia Presence Server users may want people to know that they are in a meeting and switch their phone profile to "meeting".

After switching profiles, the phone can automatically update the presence attributes in the Presence Server.

If other users wish to contact a user who has switched to the "meeting" profile they can look up the contact info in the corporate directory.

The corporate directory requests presence status from Nokia Presence Server and displays it with the usual information. For example, the profile may state that the preferred communication method is messaging.

The other users can see that the user is in a meeting, cannot read e-mails or answer the phone, and that the user prefers that any communication attempts should be done via messaging.

2.1 Environment

Figure 1 illustrates the environment in which Nokia Presence Server operates:

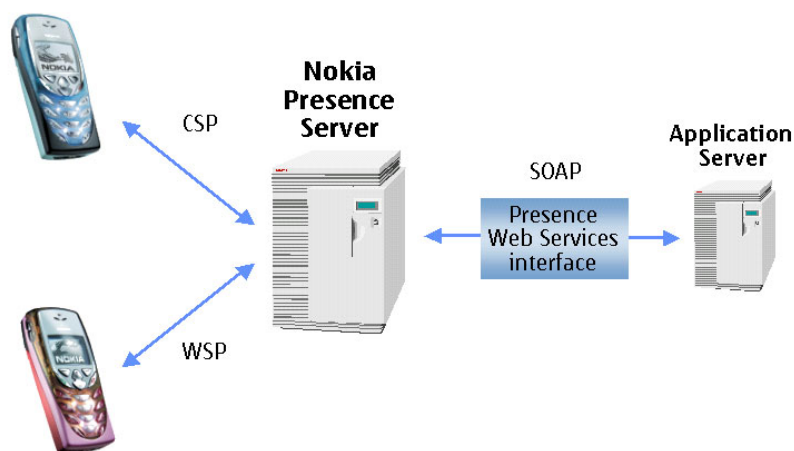


Figure 1. Overview of the Nokia Presence Server environment

- *Terminal:* There are two different types of terminals in the figure:
 1. A client that uses Client Server Protocol (CSP) for communicating with Presence Server.
 2. A client with a WAP browser that communicates over Wireless Session Protocol (WSP).
- The *Presence Web Services Interface (WSI)* provides an access for external applications, within or outside the operator's domain, to retrieve and update presence data.
- *Application Server:* Presence-enabled services are stored on servers that access Nokia Presence Server through Presence WSI.

- *Nokia Presence Server*: A software product that provides access to presence information. A cluster of servers may be used for high availability solutions. Nokia Presence Server takes care of all the tasks related to storing and distributing the presence information.

Note

The above figure contains only the components relevant to the presence service and the developer. The WAP gateway, charging server and subscriber database, for example, are not included.

2.2 General Nokia Presence Server usage model

Basic usage of the presence service consists of publishing presence information and fetching it. Both presence service users and service applications can publish and fetch information. Published presence information can be updated for various purposes.

Figure 2 illustrates the steps of the basic update flow.

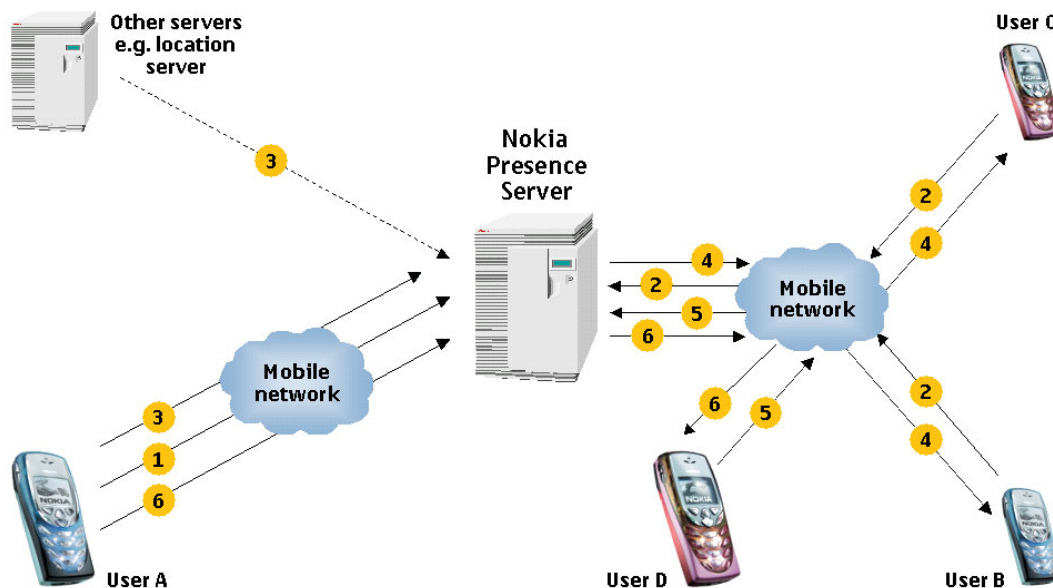


Figure 2. A simple presence update sequence with Nokia Presence Server

1. Mobile terminal user A publishes presence information on Nokia Presence Server.
2. Users B and C subscribe to user A's presence information.
3. User A can update presence information to the Nokia Presence Server by terminal, or network applications can make updates.
4. The users who have subscribed to user A's presence information receive the updated information.
5. Other users (user D) may also fetch A's presence information when needed on a one-time-basis without subscribing to updates.
6. Contact lists are used for authorisation. User A may authorise user D to subscribe to his/her private presence information.

Users can subscribe to a certain number of people, whose presence info is updated to the subscriber dynamically. For example, users can subscribe to presence information of a number of different people. Whenever one of these people updates their presence information, the subscribing user receives the updated information dynamically.

In addition to that user can fetch the presence information of other users on a one-time basis. A one-time fetch can be used to get presence information when constant updates are not necessary for the user.

2.3 Presence Server Web Service Interface versions

The Presence Web Services Interface (WSI) provides an access to external applications to retrieve and update the presence data of the users of Nokia Presence Server. For more information, refer to Chapters 4 and 5. This section briefly describes the differences between Presence WSI 1.0 and 1.1.

- In Presence WSI 1.1, applications can only subscribe to notifications that are sent to them. Applications can no longer make subscriptions which cause notifications to the terminal. For this reason, the AppNotifications flag was removed from SubscribePresenceRequest and UnsubscribePresenceRequest.
- The watcher list history feature is supported in Presence WSI 1.1. For this purpose, GetWatcherListRequest has two new optional elements, HistoryPeriod and MaxWatcherList. Also, GetWatcherListResponse two new optional elements, HistoryPeriod and WatcherTable (the UserIdTable element was removed from the response).
- ManageContactListRequest has a new optional element, ReceiveList.
- There are new datatypes, WatcherType and WatcherStatusType, to support the watcher list history feature.
- There are new error codes: 3503, 4553, 4554, 4555, and 4560.

Table 1. Presence WSI compatibility

Nokia Presence Server version	Presence WSI compatibility
1.0	Presence WSI 1.0
1.1	Presence WSI 1.0 and 1.1

3

Nokia Presence Server usage scenarios

Nokia Presence Server can be used in different ways. This chapter provides several usage scenarios. These usage scenarios should not be considered to be restrictive or mandatory in any way. Rather, these are just possible ways of using Nokia Presence Server in different environments.

These usage scenarios are described from a mobile user's point of view.

3.1 Login and Presence update

When users have subscribed to Presence service(s) from their operator, they can log in to Nokia Presence Server and update their Presence information. To accomplish this, the client terminal has to contain the following settings for Nokia Presence Server:

- Presence username
- Password
- Nokia Presence Server URL

Nokia Presence Server authenticates the user according to these settings, starts a session and the service negotiation can start:

1. The user connects to Nokia Presence Server and sends a login request with password in plain text.
2. Nokia Presence Server authenticates the user and grants access.
3. Nokia Presence Server creates a session for the user.
4. Nokia Presence Server sends a login response message to the user. This response contains the details of the login operation.

If the user is not identified by Nokia Presence Server or does not have a valid subscription to the Presence service(s), Nokia Presence Server sends the client a status message that indicates a login failure instead of the login response message. If the login is successful, an OnlineStatus presence attribute will be updated.

When logged on, users can perform other presence operations.

Presence update is possible when users have published their presence information. When the presence information of a user is updated, Nokia Presence Server notifies the users who have subscribed to that user's presence information about the changes.

Description of the Presence update sequence:

1. The user updates his/her published presence information on his/her client terminal.
2. The client sends an update request to Nokia Presence Server. This request contains only the updated attributes and their values; the omitted attributes are not modified.
3. Nokia Presence Server sends a notification message that contains the updated presence information to all users who have subscribed and have been authorised to the updating publisher's presence

To update their information, the users have to have an open session, and they have to be logged in to Nokia Presence Server.

3.2 Contact lists and Authorisation

A Contact List is a user maintained list of Nokia Presence Server users. Users can manage multiple Contact Lists for various purposes, such as a list of friends and a list of colleagues. In addition, Contact Lists are used as a mechanism for proactive presence authorisation when fetching and subscribing to presence information.

Proactive authorisation means that the subscriber is authorised beforehand. The publisher authorises certain Presence attributes to either individual users or to a group of users through a Contact List. In proactive authorisation, the actual attributes that are authorised are defined in the attribute list.

- By default, all users are proactively authorised to access the publisher's presence information contained in the default attribute list.
- After creating a Contact List, the publisher may specify a list of presence attributes to be associated with the Contact List. The list of attributes of the Contact List is accessible by all members in that Contact List.
- After adding a user to a Contact List, the publisher may specify a list of presence attributes to be associated with the user, even if the Contact List has its own attributes list.
- A block list is Contact List with an empty attribute list. Users who are included on the block list and not on any other list cannot access any presence information.

- When the user has his or her own list of attributes, the individual authorisation always has priority over Contact List's authorisation, i.e. the individual's attributes override the attributes of the Contact List.
- When a user is in multiple Contact Lists that have separate attribute lists attached, the combination of the attributes in all attribute lists are authorized to this user.

The publisher manages the contact lists and their members. Management of contact lists also includes updating or removal of presence attributes for Contact Lists and separate subscribers.

3.3 Presence query

A mobile user can fetch the presence information of another user on a one-time basis. If the user requesting the information does not appear in the target user's contact list, the default attribute list is used for authorisation. If the subscriber is authorised, he/she may access the publisher's presence information until the publisher cancels the authorisation. Proactive authorisation is explained above in section 3.2.

User 1 fetches the presence information of user 2:

1. User 1 sends a request that contains the user ID of user 2 and the list of requested presence attributes.
2. If user 1 is included in the publisher's contact lists and properly authorised, the server responds with a message that contains the authorised presence attributes associated with the requested user ID.

If the presence attribute list is missing from the request, the server sends all available presence information. The server responds to the subscriber with a message that contains the result of the request, and the authorised presence attributes.

The request message could contain more than one Contact List ID or user ID. The response message contains results for presence attribute queries for all of the requested users. Authorisation is handled separately according to each publishing user's contact lists.

Applications send fetching requests on behalf of a user. The user ID is used to identify which user the application is acting on behalf.

3.4 Presence subscription

A user is able to subscribe to another user's presence information. This means that the subscribing user will initially receive new presence information and will

always receive updated presence information when the presence information of the publishing user is updated. The publisher may give the subscriber authorisation to display only certain presence attribute values. The subscriber may unsubscribe to the presence information to stop the delivery of the publisher's presence information.

User 1 subscribes to the presence information of user 2:

1. User 1 sends a subscription request message with the parameters of user 2.
2. If user 1 is included in one of the publisher's contact lists, he/she will receive a notification of the publisher's attributes authorised for that list. The server sends a notification message that contains the requested information. Authorisation is explained above, in section 3.2.
3. Whenever the status of user 2 changes, the presence information is updated. When this happens, user 1 will receive a notification from Nokia Presence Server.
4. When user 1 no longer wants to receive the presence information, he/she may cancel the subscription by sending a request to cancel the subscription to the server.

Presence Subscription is successful regardless of whether the subscribers have been authorised or not. However, if the subscribers are not authorised for any presence attributes of the publisher, they will be authorised according to the default attribute list. If the default attribute list does not contain any Presence attributes, the subscribers will not receive any notifications.

As with a fetch request, a subscription request could contain Contact List IDs or more than one user ID. Authorisation for each subscription is handled separately according to each publishing user's contact lists.

3.5 User search

The search feature enables users to search for information belonging to other users of Nokia Presence Server. A user of Presence services may search for other users or contact lists according to their User ID substring or MSISDN.

1. The user sends a search message that contains one of the following search criteria: User ID substring or MSISDN.
2. The search results are returned to the user.

4

Presence Web Services Interface 1.0

The Presence Web Services Interface provides an access to external applications to retrieve and update the presence data of the users of Nokia Presence Server. The interface exposes presence service functionality over Web services. One of the main aims of Web services technologies is to automate business-to-business services. By offering a Web Service Interface to Service Providers, the operator enables a rapid and standards-based deployment of a wide variety of services offered by different Service providers.

The Presence Web Services Interface builds on existing and emerging technologies such as Extensible Markup Language (XML), Simple Access Object Protocol 1.1 (SOAP), Web Services Description Language 1.1 (WSDL) and Hypertext Transfer Protocol (HTTP).

The Presence Web Services Interface (WSI) provides possibilities to integrate other systems and services with Nokia Presence Server. This interface makes it possible for other applications to utilise presence information in, for example, corporate directory or scheduling and time management solutions. In addition, the Presence WSI can be used for network based or triggered presence updates and monitoring for users or other entities that may have associated presence data.

The Presence WSI is an open interface through which applications are able to use presence services. These services provide functionality for getting, updating, subscribing/unsubscribing presence information, receiving presence update notifications, using and managing contact lists and using search functions. The presence information consists of currently supported Wireless Village/OMA IMPS presence attributes (for more information, refer to Appendix A).

The Presence Web Services Interface can be used for both directions, for service initiated presence requests to the Nokia Presence Server as well as for Nokia Presence Server initiated notifications to the service applications.

Note

The Presence WSI 1.0 uses Wireless Village 1.1 attributes, while the Presence WSI 1.1 uses OMA IMPS 1.2 attributes. However, Nokia Presence Server handles standard attributes in the same way regardless of the version.

For example, if you update Wireless Village 1.1 StatusText via WSI 1.0, the update will be visible as OMA IMPS 1.2 StatusText via WSI 1.1.

4.1 Transactions

The following transactions are possible with Nokia Presence Server.

Table 2. Presence WSI transaction types

Transaction type	Description	Messages
Get Presence	A user wants to fetch the presence information of another user	Client → Server: GetPresenceRequest Server → Client: GetPresenceResponse
Update Presence Request	A user updates presence information, the server sends updated presence information to subscribers	Client → Server: UpdatePresenceRequest Server → Client: PresenceResponse
Presence Subscription	A user wants to subscribe to changes in another user's presence information	Client → Server: SubscribePresenceRequest Server → Client: PresenceResponse
Unsubscribe Request	A user wants to cancel a subscription	Client → Server: UnsubscribePresenceRequest Server → Client: PresenceResponse
Presence Notification	A Server sends a notification containing the updated presence subscriber	Server → Client: PresenceNotificationRequest Client → Server: PresenceResponse
Get Subscription List	A user gets a list of users whose presence information it is subscribed to.	Client → Server: GetSubscriptionListRequest Server → Client: GetSubscriptionListResponse
Get Watcher List	A user gets the list of users that subscribe to his/her presence attributes.	Client → Server: GetWatcherListRequest Server → Client: GetWatcherListResponse
Create Contact List	A user creates a contact list.	Client → Server: CreateContactListRequest Server → Client: PresenceResponse

DeleteContactList	A user deletes a contact list at any time.	Client → Server: DeleteContactListRequest Server → Client: PresenceResponse
GetContactLists	A user retrieves the list of all his/her own contact list IDs.	Client → Server: GetContactListsRequest Server → Client: GetContactListsResponse
ManageContactList	A user retrieves a contact list, adds/removes members, changes the name of a list, or sets the default contact list.	Client → Server: ManageContactListRequest Server → Client: ManageContactListResponse
CreateAttributeList	A user creates user or contact-list specific attribute list.	Client → Server: CreateAttributeListRequest Server → Client: PresenceResponse
DeleteAttributeList	A user deletes an attribute list from a user and/or from a contact list.	Client → Server: DeleteAttributeListRequest Server → Client: PresenceResponse
GetAttributeList	The publisher retrieves the attributes he/she associates with a specific contact list and/or user.	Client → Server: GetAttributeListRequest Server → Client: GetAttributeListResponse
Search	The client searches for data in Presence Server.	Client → Server: SearchRequest Server → Client: SearchResponse
Stop Search	The client stops the search when results are no longer required.	Client → Server: StopSearchRequest Server → Client: PresenceResponse

4.2 Data types

The following data types are included in Presence WSI 1.0. Refer to the Wireless Village specifications for more information.

Table 3. Presence WSI 1.0 data types

Name	Description	Contains
AttributeAssociationType	Contains the user ID and presence attributes associated with that user. User ID may be either a user ID or a phone number. Presence attributes are a set of WV presence attributes.	UserId (UserIdType) (Mandatory) PresenceAttributeTable (PresenceAttributeType[]) (Optional)

AttributeNameAssociationType	Contains user ID or contact list ID and presence attribute names associated with that user or contact list.	UserId (UserIdType) (Optional) ContactListId (ContactListIdType) (Optional) PresenceAttributeNameTable (PresenceAttributeNameType[]) (Optional)
ContactListIdType	Identifier for a contact list. Structure of the ID is defined in the CSP specifications; refer to [3] for more information.	STRING
ContactListPropertiesType	Properties of a contact list. Contact list can have following properties: DisplayName: a free text string given by user that can be presented in the user interface of the client. Default: flag telling if this is the default contact list.	DisplayName (STRING) (Optional) Default (BOOLEAN) (Optional)
DetailedStatusType	Helper class for holding detailed results in case of partial success. It contains the error status code and optional textual description of the status of the operation along with the ID of the user, who caused the error.	StatusCode (ErrorCodeType) (Mandatory) StatusText (STRING) (Optional) UserIdTable (UserIdType[]) (Mandatory)
ErrorCodeType	Error code describing status of the operation. It is a positive integer. The specific error value range is defined in Section 4.3.	DECIMAL
NamedValueType	Helper class for holding a presence attribute value. It holds the information of presence attribute value name and value. It may also contain nested attribute values.	Name (STRING) (Mandatory) Value (STRING) (Optional) ValueTable (NamedValueType[])(Optional)
NickNameType	A name that the client can use internally to hide the user id of contacts. It is not possible to address a NickName.	UserId (UserIdType) (Mandatory) NickName (STRING) (Mandatory)
PresenceAttributeNameType	Name of one presence attribute. It contains the name of the attribute and an optional namespace used in identification of extension attributes.	Name (STRING) (Mandatory) Namespace (STRING) (Mandatory)
PresenceAttributeType	Contains the information of one presence attribute. It contains the name of the attribute, information about the latest update time and optionally attribute validity (qualifier). The attribute values are	AttributeName (PresenceAttributeNameType) (Mandatory) UpdateTime (DATETIME) (Optional)

	(qualifier). The attribute values are stored in a specific structure.	Qualifier (BOOLEAN) (Optional) ValueTable (NamedValueType[])
SearchCriteriaType	Type for holding search criteria.	SearchElement (SearchElementType) (Mandatory) SearchString (STRING) (Mandatory)
SearchElementType	Search element type. Available search elements are: User id, MSISDN, Online status, user's first name, user's last name, user's email address, user's alias.	Enumerated STRING: USER_ID, USER_MOBILE_NUMBER, USER_ONLINE_STATUS, USER_FIRST_NAME, USER_LAST_NAME, USER_EMAIL_ADDRESS, USER_ALIAS
StatusType	Status of the operation. Contains the status code and optional textual description of the status of the operation. In case of partial success contains also detailed status for each failed user.	StatusCode (ErrorCodeType) (Mandatory) StatusText (STRING) (Optional) Details (DetailedStatusType[]) (Optional)
UserIdType	Identification of the user. It either refers to the Internet-type address or to a mobile number of the user. It is string based.	[WV:]USERNAME[@DOMAIN] MSISDN (STRING)

4.3 Error codes

The error codes of Presence WSI are listed in the table below.

Table 4. Common Error Codes

Status code	Status text	Description
1000	Success	This code indicates that the request was executed completely
1100	Partial success	This code indicates that the request was executed partially but some parts of the request could not be completed.
2001	Operation restricted	The request was refused due to lack of permission to execute the command.

2501	Invalid user ID	The specified requester user is unknown or this application is not allowed to act on behalf of the specified user.
2502	Bad parameter	The server cannot understand one of the parameters in the request.
2503	Forbidden	The server understood the request, but the principal settings prevented fulfilling the request.
2524	Invalid search ID	The server encountered an invalid search ID.
2525	Invalid search index	The server encountered an invalid search index.
3000	Server Error	The server failed to fulfill an apparently valid request.
3501	Not implemented	The server does not support the functionality required to fulfill the request.
3503	Service unavailable	The server is currently unable to handle the request due to a temporary overloading or maintenance of the server.
3504	Server timeout	The server could not produce a response within the time that it expected.
3531	Unknown user	The specified user is unknown/not valid user ID was given.
4006	Service unavailable	This indication may be sent by the server when service is temporarily unavailable, e.g. when server is busy
4007	Service denied	The client does not have permission or funds to perform the requested operation.
4500	Contact list does not exist	The contact list specified in the request does not exist.
4501	Contact list already exists	The contact list specified in the request already exists.
4502	Invalid or unsupported user properties	The user properties specified in the request are invalid or not supported.
4535	Search timed out	The server has invalidated the requested search request.
4536	Too many hits	The query returned too many hits. The client needs to narrow the query.
4537	Too broad search criteria	The search criteria are too broad. The client needs to specify more restrictive search criteria.
4550	Unknown presence attribute	The presence attribute(s) specified in the request are invalid or not supported.
4551	Unknown presence value	The presence value(s) specified in the request are invalid or not supported.
4552	Invalid or unsupported contact list property	One or more contact list properties specified in the request are invalid or not supported.

--	--	--

4.4 Operations

The following operations exist in the Presence WSI.

4.4.1 Operation #1: GetPresence

An application may get a user’s presence information at any time. The client sends GetPresenceRequest message containing the user ID or contact list ID, and optionally the list of presence requested attributes to the server. If the presence attribute list is missing from the request, it indicates to the server that all available presence information is requested.

The server responds with a GetPresenceResponse message containing the result of the request and the authorized presence attributes.

Table 5. GetPresence

Request Message Name	GetPresenceRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to get presence for as list of user ids. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to get presence for. Optional. AttributeNameTable (PresenceAttributeNameType[]) : List of requested presence attribute names. Optional.
Return Message Name	GetPresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. AttributeAssociationTable (AttributeAssociationType[]) : Contains user id and attributes associated with that user. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2510, Invalid user ID

	3000, Server error 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist 4550, Unknown presence attribute
--	--

4.4.2 Operation #2: UpdatePresence

An owner of the presence data may update presence attributes and their values by sending an UpdatePresenceRequest message to the server. The UpdatePresenceRequest contains all the changed presence attributes and their new values.

Table 6. UpdatePresence

Request Message Name	UpdatePresenceRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. PresenceAttributeTable (PresenceAttributeType[]) : A list of changed presence attribute values. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 2503, Forbidden 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4550, Unknown presence attribute

	4551, Unknown presence value
--	------------------------------

4.4.3 Operation #3: SubscribePresence

Users may subscribe to other users' presence notifications by sending the SubscribePresenceRequest message. Subscription may be targeted to specified users or contact lists, and specified presence attributes. The subscribing users receive the current status of the selected users' presence immediately and every time the attributes change. The subscribed presence information is sent in a PresenceNotificationRequest message.

Table 7. SubscribePresence

Request Message Name	SubscribePresenceRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to subscribe presence for as list of user ids. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to subscribe. Optional. AppNotifications (BOOLEAN) : Flag telling if the notifications are to be sent to the subscribing user or to the application. In Nokia Presence Server 1.1, applications can only subscribe to notifications that are sent to them, thus the element must always be true. Mandatory. AttributeNameTable (PresenceAttributeName[]) : List of requested presence attribute names. Optional. AutoSubscribe (BOOLEAN) : This feature is not supported in Nokia Presence Server 1.1, it must be set to false. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 2503, Forbidden 3000, Server error 3501, Not implemented

	3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist 4550, Unknown presence attribute
--	--

4.4.4 Operation #4: UnsubscribePresence

When applications no longer want to be notified about the presence changes, they may unsubscribe themselves by sending the UnsubscribePresenceRequest message to the server.

Table 8. UnsubscribePresence

Request Message Name	UnsubscribePresenceRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to unsubscribe as list of user ids. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to unsubscribe. Optional. AppNotifications (BOOLEAN) : Flag telling if the notifications are to be sent to the subscribing user or to the application. In Nokia Presence Server 1.1, applications can only subscribe to notifications that are sent to them, thus the element must always be true. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 2503, Forbidden 3503, Service unavailable 3504, Server timeout

	3531, Unknown user 4007, Service denied 4500, Contact list does not exist
--	---

4.4.5 Operation #5: PresenceNotification

The Presence Server sends PresenceNotificationRequest message containing the updated presence information to the application.

Table 9. PresenceNotification

Request Message Name	PresenceNotificationRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, to whom the notification is targeted to. Mandatory. AttributeAssociationTable (AttributeAssociationType[]) : The list of user and presence attribute list associations. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> -

4.4.6 Operation #6: GetSubscriptionList

The applications may get the list of users whose presence information they have subscribed to. The requesting client sends a GetSubscriptionListRequest message to the server. The server responds with a GetSubscriptionListResponse message including the list of the subscribed users.

Note

The applications can get the list of users they have subscribed to, but not which attributes have been subscribed.

Table 10. GetSubscriptionList

Request Message Name	GetSubscriptionListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory,
Return Message Name	GetSubscriptionListResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. UserIdTable (UserId[]) : List of subscribed users. Optional. ContactListIdTable (ContactListId[]) : List of contact list ids which includes those and only those contact lists which were subscribed with AutoSubscribe flag set to true. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 4007, Service denied

4.4.7 Operation #7: GetWatcherList

Applications may get the list of users that subscribe to their presence attributes. The requesting client sends a GetWatcherListRequest message to the server. The server responds with a GetWatcherListResponse message including the list of the subscribing users.

Note

As with GetSubscriptionList, the application cannot retrieve exactly what attributes every user subscribes to, but the GetAttributeListRequest can tell what attributes they are authorized to see.

Table 11. GetWatcherList

Request Message Name	GetWatcherListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory.
Return Message Name	GetWatcherListResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. UserIdTable (UserId[]) : List of subscribing users. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 4007, Service denied

4.4.8 Operation #8: CreateContactList

Users are able to create more than one contact list, but there may be system specific limitations for the maximum number of lists per user. The client sends the CreateContactListRequest message to the server including the ID of the contact list, and optionally contact list properties and a list of initial users that should be added to the list. The server creates the contact list, and responds with a PresenceResponse message.

Table 12. CreateContactList

Request Message Name	CreateContactListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. ContactListId (ContactListIdType) : Id for the contact list to be created. Mandatory. Properties (ContactListPropertiesType) : Properties for the contact list. Optional. UserNickTable (NickNameType[]) : Users to be added to the contact list. Optional.
Return Message Name	PresenceResponse()

Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 2503, Forbidden 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4501, Contact list already exists 4502, Invalid or unsupported user properties 4552, Invalid or unsupported contact list property

4.4.9 Operation #9: DeleteContactList

Users may delete a contact list at any time. The server should not unsubscribe the members implicitly, meaning that if a contact list that has been subscribed is deleted, the presence subscriptions should not be cancelled for the particular users. The client sends the DeleteContactListRequest message to the server containing the ID of the contact list to be deleted. The server removes the indicated contact list, and responds with a PresenceResponse message.

Table 13. DeleteContactList

Request Message Name	DeleteContactListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. ContactListId (ContactListIdType) : Id for the contact list to be deleted. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success

	<i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 4007, Service denied 4501, Contact list does not exist
--	--

4.4.10 Operation #10: GetContactLists

Users may retrieve a list of all their own contact list IDs. The default contact list ID is indicated in a separate information element. The client sends the GetContactListsRequest message to the server. The server replies with a GetContactListsResponse message, which contains the list of all contact list ids.

Table 14. GetContactLists

Request Message Name	GetContactListsRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory.
Return Message Name	GetContactListsResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. ContactListIdTable (ContactListIdType[]) : Table of user's contact list IDs. Optional. DefaultContactListId (ContactListId) : Default contact list ID (not in the Contact List table). Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 4007, Service denied

4.4.11 Operation #11: ManageContactList

Users may retrieve one contact list, add and remove members, change the name of the contact list, and set the default contact list. The client sends the ManageContactListRequest message to the server. The server performs the requested operation, and then replies with a ManageContactListResponse message.

In single ManageContactListRequest it is possible to specify only users to be added, users to be removed or contact list properties, but not more than one of them. The response contains either contact list users or properties, depending on which one was changed.

If the request contains only ContactListId, the server returns the properties and the users on the requested contact list.

Table 15. ManageContactList

Request Message Name	ManageContactListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. ContactListId (ContactListIdType) : Id of the contact list to be modified. Mandatory. AddNickTable (NickNameType[]) : Users to be added to the contact list. Optional. RemoveNickTable (NickNameType[]) : Users to be removed from the contact list. Optional. Properties (ContactListPropertiesType) : New properties for the contact list. Optional.
Return Message Name	ManageContactListResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. NickTable (NickName[]) : Table of users in the contact list after modification of users. Optional. Properties (ContactListPropertiesType) : The properties of the contact list after modification of properties. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable

	3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist 4502, Invalid or unsupported user properties 4552, Invalid or unsupported contact list property
--	---

4.4.12 Operation #12: CreateAttributeList

An attribute list is a set of presence attributes. The list set will define proactive authorization in the way described in the Wireless Village specification [1]. A user is able to create user or contact-list specific attribute list(s), but only one attribute lists per user or per contact list. In order to modify an attribute list, it can be overwritten by creating another one for the same user ID or contact list ID (there is no need to delete the previous list first).

Table 16. CreateAttributeList

Request Message Name	CreateAttributeListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. AttributeNameTable (PresenceAttributeName[]) : List of presence attribute names. Optional. UserIdTable (UserIdType[]) : Identifies the user(s) to be associated with this attribute list. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to be associated with this attribute list. Optional. DefaultList (BOOLEAN) : Indicates if the attributes are targeted to the default attribute list in addition to the lists specified by the fields UserIdTable and ContactListIdTable. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error

	2503, Forbidden 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist 4550, Unknown presence attribute
--	---

4.4.13 Operation #13: DeleteAttributeList

Users may delete an attribute list from a user and/or from a contact list.

Table 17. DeleteAttributeList

Request Message Name	DeleteAttributeListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to remove the attribute list association. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to remove the attribute list association. Optional. DefaultList (BOOLEAN) : Indicates if the default attribute list should be cleared. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist

4.4.14 Operation #14: GetAttributeList

The publishers may retrieve the attributes they associate with a specific contact list(s) and/or user(s), and/or the default attribute list. If no specific user(s) or contact list(s) are specified the response will include all user specific and contact list specific filters. The default attribute list is retrieved if the DefaultList element indicates “Yes”. Changing the proactive authorisation will not cancel already active subscriptions. The subscriber will not be sent notifications of attributes that are not authorised, but if it is reauthorised notifications will be sent without the need for resubscribing.

Table 18. GetAttributeList

Request Message Name	GetAttributeListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to retrieve the associated attribute list(s) for. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to retrieve the associated attribute list(s) for. Optional. DefaultList (BOOLEAN) : Indicates if the default attribute list is requested. Mandatory.
Return Message Name	GetAttributeListResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. AttributeNameAssociationTable (AttributeNameAssociationType[]) : Contains list of user ids or contact list ids and attributes associated with those users or contact lists. Optional. DefaultAttributeNameTable (PresenceAttributeNameType[]) : The list of presence attributes associated with the default attribute list. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 3531, Unknown user

	4007, Service denied
	4500, Contact list does not exist

4.4.15 Operation #15: Search

Users may search for users based on various user properties. The users may limit the number of results to be retrieved at a time, and may continue the search and go through all the results. The search is performed using a list of one or more search criteria. A search criterion consists of a search element and a search string. The search element indicates which property of the user shall be searched for the search string. When more than one search criteria are specified in the primitive, logical AND operation is assumed between the different criteria. Every search element may be present only once within the same search request. The result of a user search is always a user ID.

Table 19. Search elements for user search.

Search element	Search string
USER_ALIAS	The search string is a substring of a user's alias.
USER_EMAIL_ADDRESS	The search string is a substring of a user's e-mail address.
USER_FIRST_NAME	The search string is a substring of a user's first name.
USER_ID	The search string is a substring of a user ID.
USER_LAST_NAME	The search string is a substring of a user's last name.
USER_MOBILE_NUMBER	The search string is a mobile number.
USER_ONLINE_STATUS	The search string is the online status value.

Note

Nokia Presence Server 1.1 only supports user ID (USER_ID) and user mobile number (USER_MOBILE_NUMBER) search..

The client sends the SearchRequest message to the server including the search criteria and the search limit (maximum number of results at a time). The server responds with the SearchResponse message, which includes status of the search, and if the search was successful, it includes the search id, the search index (a continuation index to indicate where the search should be continued), the search findings (the number of items found that match the criteria so far), and the search results (the actual data).

The search result is restricted in the same way presence information is established when requested in other ways. If the searching user is not proactively authorized to see certain presence values for a user included in the search result, those presence values will not be included. If the unauthorized presence attribute is part of the search criteria, that user will not be included in the search result. This allows users that want to have certain presence attributes searchable to expose them through their default attribute list.

The user may continue the search. In this case the SearchRequest message includes only the search id and the search index. The server responds with the SearchResponse, but the message includes only the status, the search index, the search findings and the search results.

The client may modify the search index value, so that the search may be continued at a different place. The search index is valid until a new search is performed or the session ends (a previous search is invalidated when a new search is started).

Table 20. Search

Request Message Name	SearchRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. SearchCriteria (SearchCriteriaType[]) : Search criteria. Optional. SearchLimit (INTEGER) : Indicates the number of maximum search results can be received at a time. Present only in the 1st search request. Optional. SearchId (STRING) : Uniquely identifies a search transaction. The server assigns this id when the first search is performed, thus not present in the 1st search request. Optional. SearchIndex (INTEGER) : Indicates that the results shall be sent starting from this particular index. Present only when the search is continued. Optional.
Return Message Name	SearchResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. SearchId (STRING) : Uniquely identifies a search transaction. The server assigns this id when the first search is performed successfully. Optional. SearchFindings (INTEGER) : Indicates the number of the current findings. Mandatory. CompletionFlag (BOOLEAN) : Indicates whether the client can expect new results. False if the server may provide new results, true if new result will not be provided. Mandatory. SearchIndex (INTEGER) : Indicates the particular index from which the next search can start. This provides the information to the user where

	to continue the search. MandatorySearchResults (UserIdType[]) : Search results. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 2524, Invalid search ID 2525, Invalid search index 4007, Service denied 4535, Search timed out 4536, Too many hits 4537, Too broad search criteria

4.4.16 Operation #16: StopSearch

In order to indicate to the server that results are no further required from a previously issued search request, the client sends StopSearchRequest message to the server containing the search ID. The server invalidates the indicated search, and replies with a PresenceResponse message. The invalidated search ID cannot be used after invalidation.

Table 21. StopSearch

Request Message Name	StopSearchRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. SearchId (STRING) : Identifies the search to be invalidated. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID

	3000, Server error
	3503, Service unavailable
	3504, Server timeout
	2524, Invalid search ID
	4007, Service denied

4.5 Authentication and authorisation

When developing WSI applications, the following security aspects have to be taken into account:

- *Identification and authentication:* Nokia Presence Server can request a username and password from an application by using HTTP Basic or HTTP Digest Authentication. The preferred authentication method can be set per each application. When Nokia Presence Server is placing a request to an application, it can authenticate itself against HTTP Basic and HTTP Digest Authentication prompts.
- *Authorisation:* Nokia Presence Server authorises an application by comparing the credentials against Nokia Presence Server configuration information.
- *Delegation:* When using Presence WSI, an application is acting on the behalf of a Wireless Village user.

5

Presence Web Services Interface 1.1

This chapter describes the Presence Web Services Interface (WSI) 1.1. For a general description of the Presence WSI, refer to Chapter 4.

Note

The Presence WSI 1.1 uses OMA IMPS 1.2 attributes, while the Presence WSI 1.0 uses Wireless Village 1.1 attributes. However, Nokia Presence Server handles standard attributes in the same way regardless of the version.

For example, if you update Wireless Village 1.1 StatusText via WSI 1.0, the update will be visible as OMA IMPS 1.2 StatusText via WSI 1.1.

5.1 Transactions

The following transactions are possible with Nokia Presence Server.

Table 22. Presence WSI transaction types

Transaction type	Description	Messages
Get Presence	A user wants to fetch the presence information of another user	Client → Server: GetPresenceRequest Server → Client: GetPresenceResponse
Update Presence Request	A user updates presence information, the server sends updated presence information to subscribers	Client → Server: UpdatePresenceRequest Server → Client: PresenceResponse
Presence Subscription	A user wants to subscribe to changes in another user's presence information	Client → Server: SubscribePresenceRequest Server → Client: PresenceResponse
Unsubscribe Request	A user wants to cancel a subscription	Client → Server: UnsubscribePresenceRequest

		Server → Client: PresenceResponse
Presence Notification	A Server sends a notification containing the updated presence subscriber	Server → Client: PresenceNotificationRequest Client → Server: PresenceResponse
Get Subscription List	A user gets a list of users whose presence information it is subscribed to.	Client → Server: GetSubscriptionListRequest Server → Client: GetSubscriptionListResponse
Get Watcher List	A user gets the list of users that subscribe to his/her presence attributes.	Client → Server: GetWatcherListRequest Server → Client: GetWatcherListResponse
Create Contact List	A user creates a contact list.	Client → Server: CreateContactListRequest Server → Client: PresenceResponse
DeleteContactList	A user deletes a contact list at any time.	Client → Server: DeleteContactListRequest Server → Client: PresenceResponse
GetContactLists	A user retrieves the list of all his/her own contact list IDs.	Client → Server: GetContactListsRequest Server → Client: GetContactListsResponse
ManageContactList	A user retrieves a contact list, adds/removes members, changes the name of a list, or sets the default contact list.	Client → Server: ManageContactListRequest Server → Client: ManageContactListResponse
CreateAttributeList	A user creates user or contact-list specific attribute list.	Client → Server: CreateAttributeListRequest Server → Client: PresenceResponse
DeleteAttributeList	A user deletes an attribute list from a user and/or from a contact list.	Client → Server: DeleteAttributeListRequest Server → Client: PresenceResponse
GetAttributeList	The publisher retrieves the attributes he/she associates with a specific contact list. and/or user.	Client → Server: GetAttributeListRequest Server → Client: GetAttributeListResponse
Search	The client searches for data in Presence Server.	Client → Server: SearchRequest Server → Client: SearchResponse
Stop Search	The client stops the search when results are no longer required.	Client → Server: StopSearchRequest Server → Client: PresenceResponse

5.2 Data types

The following data types are included in Presence WSI 1.1. Refer to the *OMA IMPS specifications [16]* for more information.

Table 23. Presence WSI data types

Name	Description	Contains
AttributeAssociationType	Contains the user ID and presence attributes associated with that user. User ID may be either a user ID or a phone number. Presence attributes are a set of OMA IMPS presence attributes.	UserId (UserIdType) (Mandatory) PresenceAttributeTable (PresenceAttributeType[]) (Optional)
AttributeNameAssociationType	Contains user ID or contact list ID and presence attribute names associated with that user or contact list.	UserId (UserIdType) (Optional) ContactListId (ContactListIdType) (Optional) PresenceAttributeNameTable (PresenceAttributeNameType[]) (Optional)
ContactListIdType	Identifier for a contact list. Structure of the ID is defined in the CSP specification; refer to [16] for more information.	STRING
ContactListPropertiesType	Properties of a contact list. Contact list can have following properties: DisplayName: a free text string given by user that can be presented in the user interface of the client. Default: flag telling if this is the default contact list.	DisplayName (STRING) (Optional) Default (BOOLEAN) (Optional)
DetailedStatusType	Helper class for holding detailed results in case of partial success. It contains the error status code and optional textual description of the status of the operation along with the ID of the user, who caused the error.	StatusCode (ErrorType) (Mandatory) StatusText (STRING) (Optional) UserIdTable (UserIdType[]) (Mandatory)
ErrorCodeType	Error code describing status of the operation. It is a positive integer. The specific error value range is defined in Section 5.3.	DECIMAL
NamedValueType	Helper class for holding a presence attribute value. It holds the information of presence attribute	Name (STRING) (Mandatory) Value (STRING) (Optional)

	information of presence attribute value name and value. It may also contain nested attribute values.	ValueTable (NamedValueType[])(Optional)
NickNameType	A name that the client can use internally to hide the user id of contacts. It is not possible to address a NickName.	UserId (UserIdType) (Mandatory) NickName (STRING) (Optional)
PresenceAttributeType	Contains the information of one presence attribute. It contains the name of the attribute, information about the latest update time and optionally attribute validity (qualifier). The attribute values are stored in a specific structure.	AttributeName (PresenceAttributeNameType) (Mandatory) UpdateTime (DATETIME) (Optional) Qualifier (BOOLEAN) (Optional) ValueTable (NamedValueType[])
PresenceAttributeNameType	Name of one presence attribute. It contains the name of the attribute and an optional namespace used in identification of extension attributes.	Name (STRING) (Mandatory) Namespace (STRING) (Mandatory)
SearchCriteriaType	Type for holding search criteria.	SearchElement (SearchElementType) (Mandatory) SearchString (STRING) (Mandatory)
SearchElementType	Search element type. Available search elements are: User id, MSISDN, Online status, user's first name, user's last name, user's email address, user's alias.	Enumerated STRING: USER_ID, USER_MOBILE_NUMBER, USER_ONLINE_STATUS, USER_FIRST_NAME, USER_LAST_NAME, USER_EMAIL_ADDRESS, USER_ALIAS
StatusType	Status of the operation. Contains the status code and optional textual description of the status of the operation. In case of partial success contains also detailed status for each failed user.	StatusCode (ErrorCodeType) (Mandatory) StatusText (STRING) (Optional) Details (DetailedStatusType[]) (Optional)
UserIdType	Identification of the user. It either refers to the Internet-type address or to a mobile number of the user. It is string based.	[WV:]USERNAME[@DOMAIN] MSISDN (STRING)
WatcherStatusType	Watcher status. Indicates the status of the watcher. There are three possible values: User is currently subscribing - CURRENT_SUBSCRIBER,	Enumerated STRING: CURRENT_SUBSCRIBER, FORMER_SUBSCRIBER, PRESENCE_ACCESS

	Subscription ended during history period - FORMER_SUBSCRIBER, User did not subscribe, but used GetPresence to access presence info during history period - PRESENCE_ACCESS.	
WatcherType	Identifies the watchers and their status from the history period. If this element is not present at all within the response, it indicates that the server does not give any watcher information at this time. The number of these elements in getWatcherListResponse cannot be larger than MaxWatcherList value in the corresponding GetWatcherListRequest.	UserId (UserIdType) (Mandatory) WatcherStatus (WatcherStatusType) (Optional)

5.3 Error codes

The error codes of Presence WSI are listed in the table below.

Table 24. Common Error Codes

Status code	Status text	Description
1000	Success	This code indicates that the request was executed completely
1100	Partial success	This code indicates that the request was executed partially but some parts of the request could not be completed.
2001	Operation restricted	The request was refused due to lack of permission to execute the command.
2501	Invalid user ID	The specified requester user is unknown or this application is not allowed to act on behalf of the specified user.
2502	Bad parameter	The server cannot understand one of the parameters in the request.
2503	Forbidden	The server understood the request, but the principal settings prevented fulfilling the request.
2524	Invalid search ID	The server encountered an invalid search ID.
2525	Invalid search index	The server encountered an invalid search index.
3000	Server Error	The server failed to fulfill an apparently valid request.

3501	Not implemented	The server does not support the functionality required to fulfill the request.
3503	Service unavailable	The server is currently unable to handle the request due to a temporary overloading or maintenance of the server.
3504	Server timeout	The server could not produce a response within the time that it expected.
3531	Unknown user	The specified user is unknown/not valid user ID was given.
4006	Service unavailable	This indication may be sent by the server when service is temporarily unavailable, e.g. when server is busy
4007	Service denied	The client does not have permission or funds to perform the requested operation.
4500	Contact list does not exist	The contact list specified in the request does not exist.
4501	Contact list already exists	The contact list specified in the request already exists.
4502	Invalid or unsupported user properties	The user properties specified in the request are invalid or not supported.
4535	Search timed out	The server has invalidated the requested search request.
4536	Too many hits	The query returned too many hits. The client needs to narrow the query.
4537	Too broad search criteria	The search criteria are too broad. The client needs to specify more restrictive search criteria.
4550	Unknown presence attribute	The presence attribute(s) specified in the request are invalid or not supported.
4551	Unknown presence value	The presence value(s) specified in the request are invalid or not supported.
4552	Invalid or unsupported contact list property	One or more contact list properties specified in the request are invalid or not supported.
4553	Max contact lists reached	The maximum number of contact lists has been reached for the user. The server limits the maximum number of contact lists per user. The limit has been reached; so additional contact lists cannot be created. The client should not repeat the request until a contact list that belongs to the particular user has been deleted.
4554	Max contacts reached	The maximum number of contacts has been reached for the user. The server limits the maximum number of contacts per user. The limit has been reached; so

		additional contacts cannot be created. The client should not repeat the request until a contact that belongs to the particular user has been deleted.
4555	Max attribute lists reached	The maximum number of attribute lists has been reached for the user. The server limits the maximum number of attribute lists per user and the limit has been reached; so additional attribute lists cannot be created. The client should not repeat the request until an attribute list that belongs to the particular user has been deleted.
4560	Auto subscribe not supported	Automatic Subscription / Unsubscription is not supported. The server does not support the automatic subscription when adding a user to the contact list, and does not support the automatic unsubscription when deleting the contact list or removing a user from the contact list.

5.4 Operations

The following operations exist in the Presence WSI.

5.4.1 Operation #1: GetPresence

An application may get a user's presence information at any time. The client sends GetPresenceRequest message containing the user ID or contact list ID, and optionally the list of presence requested attributes to the server. If the presence attribute list is missing from the request, it indicates to the server that all available presence information is requested.

The server responds with a GetPresenceResponse message containing the result of the request and the authorized presence attributes.

Table 25. GetPresence

Request Message Name	GetPresenceRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to get presence for as list of user ids. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to get presence for. Optional. AttributeNameTable (PresenceAttributeNameType[]) : List of requested presence attribute names. Optional.

Return Message Name	GetPresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. AttributeAssociationTable (AttributeAssociationType[]) : Contains user id and attributes associated with that user. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2510, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist 4550, Unknown presence attribute

5.4.2 Operation #2: UpdatePresence

An owner of the presence data may update presence attributes and their values by sending an UpdatePresenceRequest message to the server. The UpdatePresenceRequest contains all the changed presence attributes and their new values.

Table 26. UpdatePresence

Request Message Name	UpdatePresenceRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. PresenceAttributeTable (PresenceAttributeType[]) : A list of changed presence attribute values. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success

	<i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 2503, Forbidden 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4550, Unknown presence attribute 4551, Unknown presence value
--	--

5.4.3 Operation #3: SubscribePresence

Users may subscribe to other users' presence notifications by sending the SubscribePresenceRequest message. Subscription may be targeted to specified users or contact lists, and specified presence attributes. The subscribing users receive the current status of the selected users' presence immediately and every time the attributes change. The subscribed presence information is sent in a PresenceNotificationRequest message.

Table 27. SubscribePresence

Request Message Name	SubscribePresenceRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to subscribe presence for as list of user ids. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to subscribe. Optional. AttributeNameTable (PresenceAttributeName[]) : List of requested presence attribute names. Optional. AutoSubscribe (BOOLEAN) : This feature is not supported in Nokia Presence Server 1.1, it must be set to false. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success

	1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3501, Not implemented 2503, Forbidden 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist 4550, Unknown presence attribute 4560, Auto subscribe not supported
--	---

5.4.4 Operation #4: UnsubscribePresence

When applications no longer want to be notified about the presence changes, they may unsubscribe themselves by sending the UnsubscribePresenceRequest message to the server.

Table 28. UnsubscribePresence

Request Message Name	UnsubscribePresenceRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to unsubscribe as list of user ids. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to unsubscribe. Optional.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID

	2503, Forbidden 3000, Server error 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist
--	---

5.4.5 Operation #5: PresenceNotification

The Presence Server sends PresenceNotificationRequest message containing the updated presence information to the application.

Table 29. PresenceNotification

Request Message Name	PresenceNotificationRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, to whom the notification is targeted to. Mandatory. AttributeAssociationTable (AttributeAssociationType[]) : The list of user and presence attribute list associations. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> -

5.4.6 Operation #6: GetSubscriptionList

The applications may get the list of users whose presence information they have subscribed to. The requesting client sends a GetSubscriptionListRequest message to the server. The server responds with a GetSubscriptionListResponse message including the list of the subscribed users.

Note

The applications can get the list of users they have subscribed to, but not which attributes have been subscribed.

Table 30. GetSubscriptionList

Request Message Name	GetSubscriptionListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory,
Return Message Name	GetSubscriptionListResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. UserIdTable (UserId[]) : List of subscribed users. Optional. ContactListIdTable (ContactListId[]) : List of contact list ids which includes those and only those contact lists which were subscribed with AutoSubscribe flag set to true. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 4007, Service denied

5.4.7 Operation #7: GetWatcherList

Applications may get the list of users that subscribe to their presence attributes. The requesting client sends a GetWatcherListRequest message to the server. The server responds with a GetWatcherListResponse message including the list of the subscribing users.

Note

As with GetSubscriptionList, the application cannot retrieve exactly what attributes every user subscribes to, but the GetAttributeListRequest can tell what attributes they are authorized to see.

Table 31. GetWatcherList

Request Message Name	GetWatcherListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. HistoryPeriod (INTEGER) : Indicates the time period in seconds on the longest possible history of the watcher (from the time of request) that should be returned. In case of absence, it indicates the user requests the watcher list at the time of the request only. The value 0 must not be used. Optional. MaxWatcherList (INTEGER) : Indicates the maximum number of Watcher elements in GetWatcherListResponse. Optional.
Return Message Name	GetWatcherListResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. HistoryPeriod (INTEGER) : Indicates the time period in seconds in which the watcher history has been accumulated. The returned value is not larger than the requested period. Absence indicates that the Watcher information element returns only the current subscribers at the time of the request even if the history log is requested. Optional. WatcherTable (Watcher[]) : Identifies the watchers and their status from the history period. If this element is not present at all within the response, it indicates that the server does not give any watcher information at this time. The number of this element in GetWatcherListResponse is not larger than Max Watcher List value in the corresponding GetWatcherListRequest. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 4007, Service denied

5.4.8 Operation #8: CreateContactList

Users are able to create more than one contact list, but there may be system specific limitations for the maximum number of lists per user. The client sends the CreateContactListRequest message to the server including the ID of the contact list, and optionally contact list properties and a list of initial users that

should be added to the list. The server creates the contact list, and responds with a PresenceResponse message.

Table 32. CreateContactList

Request Message Name	CreateContactListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. ContactListId (ContactListIdType) : Id for the contact list to be created. Mandatory. Properties (ContactListPropertiesType) : Properties for the contact list. Optional. UserNickTable (NickNameType[]) : Users to be added to the contact list. Optional.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 2503, Forbidden 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4501, Contact list already exists 4502, Invalid or unsupported user properties 4552, Invalid or unsupported contact list property 4553, Max contact lists reached 4554, Max contacts reached

5.4.9 Operation #9: DeleteContactList

Users may delete a contact list at any time. The server should not unsubscribe the members implicitly, meaning that if a contact list that has been subscribed is deleted, the presence subscriptions should not be cancelled for the particular users. The client sends the DeleteContactListRequest message to the server containing the ID of the contact list to be deleted. The server removes the indicated contact list, and responds with a PresenceResponse message.

Table 33. DeleteContactList

Request Message Name	DeleteContactListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. ContactListId (ContactListIdType) : Id for the contact list to be deleted. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 4007, Service denied 4501, Contact list does not exist

5.4.10 Operation #10: GetContactLists

Users may retrieve a list of all their own contact list IDs. The default contact list ID is indicated in a separate information element. The client sends the GetContactListsRequest message to the server. The server replies with a GetContactListsResponse message, which contains the list of all contact list ids.

Table 34. GetContactLists

Request Message Name	GetContactListsRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory.
Return Message Name	GetContactListsResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. ContactListIdTable (ContactListIdType[]) : Table of user's contact list IDs. Optional. DefaultContactListId (ContactListId) : Default contact list ID (not in the Contact List table). Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 4007, Service denied

5.4.11 Operation #11: ManageContactList

Users may retrieve one contact list, add and remove members, change the name of the contact list, and set the default contact list. The client sends the ManageContactListRequest message to the server. The server performs the requested operation, and then replies with a ManageContactListResponse message.

In single ManageContactListRequest it is possible to specify only users to be added, users to be removed or contact list properties, but not more than one of them. The response contains contact list properties if the request does not contain users to be added or removed. The response contains users on the requested contact list if the ReceiveList flag in the request is set to true.

Table 35. ManageContactList

Request Message Name	ManageContactListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. ContactListId (ContactListIdType) : Id of the contact list to be modified. Mandatory. AddNickTable (NickNameType[]) : Users to be added to the contact list. Optional. RemoveNickTable (NickNameType[]) : Users to be removed from the contact list. Optional. Properties (ContactListPropertiesType) : New properties for the contact list. Optional. ReceiveList (BOOLEAN) : Indicates if the client wants to receive the NickTable in the ListManageResponse. Mandatory.
Return Message Name	ManageContactListResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. NickTable (NickName[]) : Table of users in the contact list after modification of users. Optional. Properties (ContactListPropertiesType) : The properties of the contact list after modification of properties. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist 4502, Invalid or unsupported user properties 4552, Invalid or unsupported contact list property 4554, Max contacts reached

5.4.12 Operation #12: CreateAttributeList

An attribute list is a set of presence attributes. The list set will define proactive authorization in the way described in the Wireless Village and OMA IMPS specifications. A user is able to create user or contact-list specific attribute list(s), but only one attribute lists per user or per contact list. In order to modify an attribute list, it can be overwritten by creating another one for the same user ID or contact list ID (there is no need to delete the previous list first).

Table 36. CreateAttributeList

Request Message Name	CreateAttributeListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. AttributeNameTable (PresenceAttributeName[]) : List of presence attribute names. Optional. UserIdTable (UserIdType[]) : Identifies the user(s) to be associated with this attribute list. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to be associated with this attribute list. Optional. DefaultList (BOOLEAN) : Indicates if the attributes are targeted to the default attribute list in addition to the lists specified by the fields UserIdTable and ContactListIdTable. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 2503, Forbidden 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist 4550, Unknown presence attribute

5.4.13 Operation #13: DeleteAttributeList

Users may delete an attribute list from a user and/or from a contact list.

Table 37. DeleteAttributeList

Request Message Name	DeleteAttributeListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to remove the attribute list association. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to remove the attribute list association. Optional. DefaultList (BOOLEAN) : Indicates if the default attribute list should be cleared. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist

5.4.14 Operation #14: GetAttributeList

The publishers may retrieve the attributes they associate with a specific contact list(s) and/or user(s), and/or the default attribute list. If no specific user(s) or contact list(s) are specified the response will include all user specific and contact list specific filters. The default attribute list is retrieved if the DefaultList element indicates “Yes”. Changing the proactive authorisation will not cancel already active subscriptions. The subscriber will not be sent

notifications of attributes that are not authorised, but if it is reauthorised notifications will be sent without the need for resubscribing.

Table 38. GetAttributeList

Request Message Name	GetAttributeListRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. UserIdTable (UserIdType[]) : Identifies the user(s) to retrieve the associated attribute list(s) for. Optional. ContactListIdTable (ContactListIdType[]) : Identifies the contact list(s) to retrieve the associated attribute list(s) for. Optional. DefaultList (BOOLEAN) : Indicates if the default attribute list is requested. Mandatory.
Return Message Name	GetAttributeListResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. AttributeNameAssociationTable (AttributeNameAssociationType[]) : Contains list of user ids or contact list ids and attributes associated with those users or contact lists. Optional. DefaultAttributeNameTable (PresenceAttributeNameType[]) : The list of presence attributes associated with the default attribute list. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success 1100, Partial success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 3531, Unknown user 4007, Service denied 4500, Contact list does not exist

5.4.15 Operation #15: Search

Users may search for users based on various user properties. The users may limit the number of results to be retrieved at a time, and may continue the search and go through all the results. The search is performed using a list of one or more search criteria. A search criterion consists of a search element and a search string. The search element indicates which property of the user shall be searched for the search string. When more than one search criteria are specified in the primitive, logical AND operation is assumed between the different criteria. Every search element may be present only once within the same search request. The result of a user search is always a user ID.

Table 39. Search elements for user search.

Search element	Search string
USER_ALIAS	The search string is a substring of a user's alias.
USER_EMAIL_ADDRESS	The search string is a substring of a user's e-mail address.
USER_FIRST_NAME	The search string is a substring of a user's first name.
USER_ID	The search string is a substring of a user ID.
USER_LAST_NAME	The search string is a substring of a user's last name.
USER_MOBILE_NUMBER	The search string is a mobile number.
USER_ONLINE_STATUS	The search string is the online status value.

Note

Nokia Presence Server 1.1 only supports user ID (USER_ID) and user mobile number (USER_MOBILE_NUMBER) search.

The client sends the SearchRequest message to the server including the search criteria and the search limit (maximum number of results at a time). The server responds with the SearchResponse message, which includes status of the search, and if the search was successful, it includes the search id, the search index (a continuation index to indicate where the search should be continued), the search findings (the number of items found that match the criteria so far), and the search results (the actual data).

The search result is restricted in the same way presence information is established when requested in other ways. If the searching user is not proactively authorized to see certain presence values for a user included in the search result, those presence values will not be included. If the unauthorized

presence attribute is part of the search criteria, that user will not be included in the search result. This allows users that want to have certain presence attributes searchable to expose them through their default attribute list.

The user may continue the search. In this case the SearchRequest message includes only the search id and the search index. The server responds with the SearchResponse, but the message includes only the status, the search index, the search findings and the search results.

The client may modify the search index value, so that the search may be continued at a different place. The search index is valid until a new search is performed or the session ends (a previous search is invalidated when a new search is started).

Table 40. Search

Request Message Name	SearchRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. SearchCriteria (SearchCriteriaType[]) : Search criteria. Optional. SearchLimit (INTEGER) : Indicates the number of maximum search results can be received at a time. Present only in the 1st search request. Optional. SearchId (STRING) : Uniquely identifies a search transaction. The server assigns this id when the first search is performed, thus not present in the 1st search request. Optional. SearchIndex (INTEGER) : Indicates that the results shall be sent starting from this particular index. Present only when the search is continued. Optional.
Return Message Name	SearchResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory. SearchId (STRING) : Uniquely identifies a search transaction. The server assigns this id when the first search is performed successfully. Optional. SearchFindings (INTEGER) : Indicates the number of the current findings. Mandatory. CompletionFlag (BOOLEAN) : Indicates whether the client can expect new results. False if the server may provide new results, true if new result will not be provided. Mandatory. SearchIndex (INTEGER) : Indicates the particular index from which the next search can start. This provides the information to the user where to continue the search. Mandatory SearchResults (UserIdType[]) : Search results. Optional.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success

	<i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout 2524, Invalid search ID 2525, Invalid search index 4007, Service denied 4535, Search timed out 4536, Too many hits 4537, Too broad search criteria
--	--

5.4.16 Operation #16: StopSearch

In order to indicate to the server that results are no further required from a previously issued search request, the client sends StopSearchRequest message to the server containing the search ID. The server invalidates the indicated search, and replies with a PresenceResponse message. The invalidated search ID cannot be used after invalidation.

Table 41. StopSearch

Request Message Name	StopSearchRequest()
Request Information Elements	UserId (UserIdType) : Identifies the user, who is the requester. Mandatory. SearchId (STRING) : Identifies the search to be invalidated. Mandatory.
Return Message Name	PresenceResponse()
Return Information Elements	Status (StatusType) : Status of the operation. Mandatory.
Return Status (StatusType)	<i>Application responses:</i> 1000, Success <i>Fault responses:</i> 2501, Invalid user ID 3000, Server error 3503, Service unavailable 3504, Server timeout

	2524, Invalid search ID 4007, Service denied
--	---

5.5 Authentication and authorisation

For information about authentication and authorisation, refer to Section 4.5.

6

Creating a WSI-enabled service

This chapter contains step-by-step instructions on how to create a Presence WSI application using Axis tool [6] and J2SE [7]. Even though this chapter demonstrates usage of Axis, it is possible to create Presence WSI application with any SOAP 1.1 / WSDL 1.1 compliant tool.

As a precondition this chapter assumes that both Axis and Java™ 2 SDK have been downloaded and installed as instructed in [6] and [7].

6.1 Required information

In order to access presence service through Presence WSI, you need to obtain following information from your presence service provider.

- User ID to be used by your application. In order to be able to publish presence information or request other users presence information, your application needs a user ID it uses when it sends requests through Presence WSI.
- HTTP authentication username for your application.
- HTTP authentication password for your application.
- URL to be used by your application for accessing Presence WSI.

In the WSI application example created in this chapter we use following values:

- User ID: `wv:john.smith@domain.com`
- HTTP authentication username: `pswsiuser`
- HTTP authentication password: `pswsipwd`
- Presence WSI 1.0 URL:
`http://host:19080/axis/services/PSWSIPort`

Note

The Presence WSI 1.0 URL is:

`http://host:19080/axis/services/PSWSIPort.`

The Presence WSI 1.1 URL is:

`http://host:19080/axis/services/PSWSIPort11.`

6.2 Creating a WSI application

Presence WSI service description consists of four files:

- `PresenceWSI.wsdl`: Presence WSI WSDL description.
- `PresenceWSI-callback.wsdl`: Template for WSDL description for the WSI application callback interface.
- `PresenceCommon.wsdl`: Common WSDL definitions for presence WSI.
- `PresenceWSI.xsd`: XML schema for presence WSI.

**To create a WSI application**

1. Ensure that that Axis [6] Java™ 2 SDK [7] and are properly installed and that Axis libraries are included in the CLASSPATH environment variable.
2. Use Axis WSDL -> Java tool to generate Java client bindings from presence WSDL description:

```
java org.apache.axis.wsdl.WSDL2Java -p <dest java package> -o <dest dir> PresenceWSI.wsdl
```

Note

For further information on WSDL2Java tool see Axis documentation [6].

3. To access presence service, write and compile an application which uses Java classes generated in step 2.

6.3 Sample WSI application

This chapter demonstrates how to perform the tasks mentioned above in practise.

Java client bindings are generated to directory
src/com/wsiappdev/presence by running the command:

```
java org.apache.axis.wsdl.WSDL2Java -p
com.wsiappdev.presence -o src PresenceWSI.wsdl
```

Below is a simple application, which acts with user ID
john.smith@domain.com and fetches OnlineStatus attribute of user
alice.smith@domain.com.

Note

The sample application uses Presence WSI 1.0.

```
/*****
```

```
PresenceFetchClient
```

```
This is a simple client designed to show how to fetch a users
presence attribute from the Nokia Presence Server. It uses classes
generated by Apache Axis using WSDL2Java and the Nokia Presence
Server WSI description.
```

```
*****/
```

```
package com.wsiappdev.sampleclient;
```

```
import com.wsiappdev.presence.*;          // Generated classes.
import java.net.URL;
import org.apache.axis.client.Service;    // Axis libraries.
import org.apache.axis.*;                // Axis libraries.
```

```
public class PresenceFetchClient
{
```

```
    /* ----- PUBLIC ----- */
```

```
    /**
     * Constructor
     */
```

```
    public PresenceFetchClient()
    {
    }
```

```
    /**
     * Main method.
     */
```

```
    public static void main(String[] args)
    {
        // This is the object which will perform the presence fetch.
        PresenceFetchClient pfc = new PresenceFetchClient();

        // To make a request to the PS, there are 3 steps:
        // 1. Create the request message.
        // 2. Send the message.
```

```

// 3. Analyse the response to get the attribute we want.

// 1. Create the request message.
// We will do something simple: ask for user's OnlineStatus.
String watcherId = "wv:john.smith@domain.com";
String presentityId = "wv:alice.smith@domain.com";
String attribute = "OnlineStatus";

System.out.println("1. Create the request message");
GetPresenceRequestType req =
    pfc.createPresenceRequestMessage(
        watcherId, presentityId, attribute);

// 2. Send the message.
String urlString =
    "http://host:19080/axis/services/PSWSIPort";
String authId = "pswsiuser";
String authPwd = "pswsipwd";

System.out.println("2. Send the message");
GetPresenceResponseType res =
    pfc.sendMessage(req, urlString, authId, authPwd);

// 3. Analyse the response to get the attribute we want.
System.out.println(
    "3. Analyse the response to get the attribute we want");
String onlineStatus = pfc.analysePresenceResponseMessage(res);
if (onlineStatus != null)
{
    System.out.println(
        "OnlineStatus for " + presentityId +
        " is " + onlineStatus);
}
}

/* ----- PRIVATE ----- */

/**
 * This method creates the GetPresenceRequestType message for
 * sending to the Nokia Presence Server. It creates a simple
 * message designed to fetch a single presence attribute for a
 * single presentity. The method also takes a user id of the
 * watcher. This is used by the Nokia Presence Server for
 * authorisation of the request, i.e. that the requested
 * presentity authorises that this particular watcher is
 * allowed to access this attribute value.
 *
 * @param watcherId user id of the watcher
 * @param presentityId user id of the requested user
 * @param presenceAttribute name of the requested presence attribute
 * @return message to be sent to the presence WSI
 */
private GetPresenceRequestType createPresenceRequestMessage(
    String watcherId, String presentityId, String presenceAttribute)

```

```

{
    // Put the presentity in the requested users table.
    UserIdType[] pis = { new UserIdType(presentityId) };
    UserIdTableType pit = new UserIdTableType();
    pit.setUserId(pis);

    // Put the requested presence attribute in the attribute table.
    PresenceAttributeNameType pan = new PresenceAttributeNameType();
    pan.setName(presenceAttribute);
    PresenceAttributeNameType[] pans = { pan };
    AttributeNameTableType pant = new AttributeNameTableType();
    pant.setAttributeName(pans);

    // Build the get presence request type.
    GetPresenceRequestType gpreq = new GetPresenceRequestType();
    gpreq.setUserId(new UserIdType(watcherId)); // watcher
    gpreq.setUserIdTable(pit);                // presentities
    gpreq.setAttributeNameTable(pant);         // presence attributes

    return gpreq;
}

/**
 * This method sends a GetPresenceRequestType message to a URI
 * identified by http://host:port/file. The method also takes an
 * appId and appPwd strings used to identify the application to
 * Nokia Presence Server using HTTP Authentication.
 *
 * @param req message to be sent to presence WSI
 * @param urlString presence WSI URL string
 * @param authId HTTP authentication username for this application
 * @param authPwd HTTP authentication password for this application
 * @return message received as a response from presence WSI
 */
private GetPresenceResponseType sendMessage(
    GetPresenceRequestType req, String urlString,
    String authId, String authPwd)
{
    GetPresenceResponseType res = null;
    try
    {
        // Send the message through the stub.
        URL url = new URL(urlString);
        PresenceBindingStub stub =
            new PresenceBindingStub(url, new Service());
        stub.setUsername(authId);
        stub.setPassword(authPwd);
        res = stub.getPresence(req);
    }
    catch (AxisFault af)
    {
        System.out.println("Axis faultCode: "
            + af.getFaultCode()
            + ", faultString: "

```

```

        + af.getFaultString()
        + ", faultActor: "
        + af.getFaultActor());
    af.printStackTrace();
}
catch (Exception e)
{
    e.printStackTrace();
}
return res;
}

/**
 * This method analyses the response message and returns the first
 * attribute value. It is assumed that the response message
 * corresponds to the request message and therefore will only
 * have a single attribute.
 *
 * @param res get presence response message
 * @return value of the first presence attribute in given response
 * message, or null if no response message was given
 */
private String analysePresenceResponseMessage(
    GetPresenceResponseType res)
{
    if (res != null)
    {
        StatusType st = res.getStatus();
        ErrorCodeType stErrCode = st.getStatusCode();
        System.out.println("Result Code: " + stErrCode.getValue());
        System.out.println("Result Status: " + st.getStatusText());

        AttributeAssociationTableType aat =
            res.getAttributeAssociationTable();
        AttributeAssociationType[] aas =
            aat.getAttributeAssociation();
        System.out.println("Result UserId: " + aas[0].getUserId());

        PresenceAttributeTableType pat =
            aas[0].getPresenceAttributeTable();
        PresenceAttributeType[] pas = pat.getPresenceAttribute();
        ValueTableType vt= pas[0].getValueTable();
        NamedValueType[] nvs = vt.getNamedValue();
        System.out.println(
            "Result Name: " + pas[0].getAttributeName().getName() +
            " (" + pas[0].getAttributeName().getNamespace() + ")");
        System.out.println(
            "Result Qualifier: " + pas[0].getQualifier());
        System.out.println(
            "Result UpdateTime: " +
            pas[0].getUpdateTime().getTime());
        System.out.println("Result Value: " + nvs[0].getValue());
        return nvs[0].getValue();
    }
}

```

```
        else
        {
            System.out.println("No result returned.");
        }
        return null;
    }
}
```

The application is compiled with command:

```
javac -d classes
src\com\wsiappdev\sampleclient\PresenceFetchClient.java
src\com\wsiappdev\presence\*.java
```

When directory containing compiled application classes is added to the CLASSPATH environment variable the application can be run with command:

```
java com.wsiappdev.sampleclient.PresenceFetchClient
```

Below is an example of the output produced by the application.

```
1. Create the request message
2. Send the message
3. Analyse the response to get the attribute we want
Result Code: 1000
Result Status: Operation was successful
Result UserId: wv:alice.smith@domain.com
Result Name: OnlineStatus (http://www.wireless-village.org/PA1.1)
Result Qualifier: true
Result UpdateTime: Fri Jan 03 14:26:13 EET 2003
Result Value: T
OnlineStatus for wv:alice.smith@domain.com is T
```

6.4 Receiving presence notifications to WSI applications

To enable receiving of presence notifications to a Presence WSI application, the application developer needs to implement a WSI interface defined in PresenceWSI-callback.wsdl. Implementation of a WSI requires a web server at the application side. For example, Tomcat [8] is a free servlet engine/Web server that can be used for this purpose.

**To create a presence notification listener application**

1. Modify location of PresenceCallbackWSI service in PresenceWSI-callback.wsdl to be URL your application listens to.
2. Use Axis WSDL -> Java tool to generate Java classes which take care of the mapping of the SOAP protocol and Java:

```
java org.apache.axis.wsdl.WSDL2Java -p <dest java  
package> -o <dest dir> -s PresenceWSI-callback.wsdl
```

3. Write and compile your notification handling application. Use the generated PresenceCallbackBindingImpl.java template as a basis for your notification handling code.
4. Deploy Axis servlet to your Web server (how to do this varies on product by product basis) and make your application classes available to it.
5. Deploy the application to your Axis server (Axis server needs to be running at this point) with Axis AdminClient tool. Use deploy.wsdd generated at step 2 and Axis AdminClient as follows:

```
java org.apache.axis.client.AdminClient -  
lhttp://host:port/axis/services/AdminService  
deploy.wsdd
```

With the `-l` parameter we specified the URL of the AdminService where the AdminClient should connect. Normally this should be on the localhost. With `-u` and `-p` parameters we can specify username and password for HTTP Basic authentication.

Note

The presence service provider needs to register the presence notification listener URL for your application to the Nokia Presence Server before the application will receive any presence notifications.

Appendix A: Presence attributes

General Structure of Presence Attributes

Following table shows the overall structure of presence attributes. These attributes consist of the currently supported Wireless Village/OMA IMPS attributes. The Wireless Village and OMA IMPS Presence Attributes can be found in [2] and [16].

Table 42. Structure of presence attributes

Information Element	Req	Single/Multiple	Type	Description
AttributeName	M	S	Enum. String	Name of the attribute
UpdateTime	O	S	Date	Last update time of the attribute.
Qualifier	O	S	Boolean	Validity of ValueTable field
ValueTable	O	M	Multiple types	Value of the attribute

AttributeName

Each presence attribute has a Name. The Names of the attributes are defined in this document and are shown in the table below.

Table 43. Names of presence attributes

Attribute Name	Description
OnlineStatus	Shows if the client device is logged on a server
Registration	Shows if the client device is registered in mobile network
ClientInfo	Information about client
TimeZone	Local time zone of the client device
GeoLocation	Geographical location of the client device
Address	Address of the client device
FreeTextLocation	Free text description of the location of the user
PLMN	PLMN code of the network the client device is registered to
CommCap	Communication capabilities of the client
UserAvailability	Availability of the user for communication
PreferredContacts	Contact preferences of the user
PreferredLanguage	Language preference of the user

StatusText	User specified status text
StatusMood	Mood of the user
Alias	Alias name for the user
StatusContent	Media info for user status
ContactInfo	A vCard for the user
InfoLink	A set of URLs that the user has selected as extra information. Introduced in OMA IMPS 1.2. Available only in Presence WSI 1.1.

Note

The attribute names and value names are case sensitive.

Enumerated values are case insensitive.

UpdateTime

The UpdateTime field indicates when the attribute has been updated the last time. This parameter is optional when a client makes presence update.

Table 44. UpdateTime

Field name	UpdateTime
Data type	Date
Format	CCYY-MM-DDThh:mm:ss, where "CC" represents the century, "YY" the year, "MM" the month and "DD" the day. The letter "T" is the date/time separator and "hh", "mm", "ss" represent hour, minute and second respectively [12].
Description	Last update time of the attribute.
Range	-

Qualifier

The Qualifier field indicates if an attribute contains valid value. An attribute value is invalid when:

- An attribute is authorized but not yet updated for the first time
- The user wants to indicate that the value of the attribute is unknown.

Table 45. Qualifier

Field name	Qualifier
Data type	Boolean
Format	Following values: T – The Value part of the attribute contains valid information F – The Value part of the attribute is unknown or undetermined. Any previous value stored in client terminal is out-of-date.
Description	The validity of the Value field of an attribute
Range	T, F

The server passes the Qualifier to the requesting or subscribing clients. A change in Qualifier is treated same way as a change in Value, i.e. it may trigger a notification. The Qualifier value has impact on the server behavior for server-originated attributes.

For example, if the GeoLocation attribute for a publisher is agreed to be server originated and the publisher sends a client originated update with Qualifier indicating the value to be invalid, then the server shall accept the new Qualifier and shall keep it in the invalid state until the publisher client sends a new update for this attribute with Qualifier indicating that the value is valid.

The attribute value fields are described in the following sections.

OnlineStatus

This indicates whether the client application is logged on to the Wireless Village server or not. This attribute is server maintained, a client cannot update the value. A client can update the qualifier.

Defined information elements are:

Table 46. OnlineStatus

AttributeName.Name	OnlineStatus
ValueTable.Name	PresenceValue
ValueTable.Value	Following values: T – At least one client application of the user is logged on to the server F – No client application of the user is logged on to the server
Description	The login status of the client
Range	T, F

Registration

This information indicates the registration status of the client in the network.

Defined information elements are:

Table 47. Registration

AttributeName.Name	Registration
ValueTable.Name	PresenceValue
ValueTable.Value	Following values: T – The client is registered in mobile network F – the client is not registered in mobile network or out of coverage
Description	The registration status of the client
Range	T, F

ClientInfo

This indicates to the subscriber information about this particular client. This information includes manufacturer, model, a user defined name, the current language and a description of the client.

Defined information elements are:

Table 48. ClientInfo

AttributeName.Name	ClientInfo	Req	Description
ValueTable.Name	ClientType	O	Describes the client type, if PC, Mobile, HTML etc.
ValueTable.Name	DevManufacturer	O	Name of the device manufacture
ValueTable.Name	ClientProducer	O	Name of the producer of the client application
ValueTable.Name	Model	O	Model of the client device
ValueTable.Name	ClientVersion	O	Version of the client application
ValueTable.Name	Language	O	Language setting of the client device

Table 49. ClientType

ValueTable.Name	ClientType
ValueTable.Value	One of the following values: MOBILE_PHONE COMPUTER PDA CLI OTHER
Description	Type of the client
Range	MOBILE_PHONE, COMPUTER, PDA, CLI, OTHER

Table 50. DevManufacturer

ValueTable.Name	DevManufacturer
ValueTable.Value	Free text
Description	Name of the device manufacturer
Range	-

Table 51. ClientProducer

ValueTable.Name	ClientProducer
ValueTable.Value	Free text
Description	Name of the producer of the client application
Range	-

Table 52. Model

ValueTable.Name	Model
ValueTable.Value	Free text
Description	Model of the client terminal
Range	-

Table 53. ClientVersion

ValueTable.Name	ClientVersion
ValueTable.Value	Free text
Description	Version of the client application
Range	-

Table 54. Language

ValueTable.Name	Language
ValueTable.Value	ISO 639-2/T (three letter language code)
Description	Language setting of the client terminal
Range	-

TimeZone

This gives the local time of the publisher.

Defined information elements are:

Table 55. TimeZone

AttributeName.Name	TimeZone
ValueTable.Name	Zone
ValueTable.Value	Offset of the local time from the UTC expressed in format defined by ISO 8601 in basic format Example: a positive difference of two hours between local time and the UTC time is given as +0200 or simply +02
Description	This is the time zone of the publisher.
Range	-

GeoLocation

This gives the measured position of the client device. The measurements may be either sensor based (e.g. GPS) or network based or combination of both. The attribute can be originated from the publisher server or from the client device.

The attribute Accuracy gives indication of the average positioning accuracy achieved by the method. The content includes at least the latitude/longitude lateral position but may include also the vertical position.

Defined information elements are:

Table 56. GeoLocation

AttributeName.Name	GeoLocation	Req	Description
ValueTable.Name	Longitude	M	Longitude
ValueTable.Name	Latitude	M	Latitude
ValueTable.Name	Altitude	O	Altitude
ValueTable.Name	Accuracy	O	Accuracy of location information

Table 57. Longitude

ValueTable.Name	Longitude
ValueTable.Value	LL_format DMS3 as described in [13] with the restriction that the output direction indicator can be only in the end of the string and can only have values (W E). For example: 35 24 15.652W indicates 35 degrees 24 minutes and 15.652 seconds longitude west.
Description	The longitudinal position defined as the angle between a reference plane and a plane passing through the point with both planes being perpendicular to the equatorial plane. The datum is WGS-84.
Range	-

Table 58. Latitude

ValueTable.Name	Latitude
ValueTable.Value	LL_format DMS3 as described in [13] with the restriction that the output direction indicator can be only in the end of the string and can only have values (N S). For example: 12 36 22.5N indicates 12 degrees 36 minutes and 22.5 seconds latitude north.
Description	The latitude position defined as angle from the equatorial plane to the vertical direction of a line passing through the point and normal to the reference ellipsoid (WGS-84).
Range	-

Table 59. Altitude

ValueTable.Name	Altitude
ValueTable.Value	Integer
Description	The altitude of a point in meters in respect of the ellipsoid which is used to be define the coordinates.
Range	-

Table 60. Accuracy

ValueTable.Name	Accuracy
ValueTable.Value	Integer
Description	This is the accuracy of location information in meters. It represents the biggest uncertainty in supplied data (either longitude, latitude or altitude).
Range	-

Address

This attribute gives the location of the user in a human understandable text form such as the address (street, city, state), or highway, or community etc.

Defined information elements are:

Table 61. Address

AttributeName.Name	Address	Req	Description
ValueTable.Name	Country	O	Country
ValueTable.Name	City	O	City
ValueTable.Name	Street	O	Street
ValueTable.Name	Crossing1	O	Crossing
ValueTable.Name	Crossing2	O	Crossing
ValueTable.Name	Building	O	Building
ValueTable.Name	NamedArea	O	Named area
ValueTable.Name	Accuracy	O	Accuracy

Table 62. Country

ValueTable.Name	Country
ValueTable.Value	The two letter Alpha-2 format as defined in the ISO 3166-1 specification
Description	This is the country of the current user location
Range	-

Table 63. City

ValueTable.Name	City
ValueTable.Value	String
Description	This is the city of the current user location
Range	-

Table 64. Street

ValueTable.Name	Street
ValueTable.Value	String
Description	This is the street of the current user location
Range	-

Table 65. Crossing1

ValueTable.Name	Crossing1
ValueTable.Value	String
Description	This is the first street in a crossing of two streets
Range	-

Table 66. Crossing2

ValueTable.Name	Crossing2
ValueTable.Value	String
Description	This is the second street in a crossing of two streets
Range	-

Table 67. Building

ValueTable.Name	Building
ValueTable.Value	String
Description	This is a building name or number for current user location
Range	-

Table 68. NamedArea

ValueTable.Name	NamedArea
ValueTable.Value	String
Description	This is any named area for current user location
Range	-

Table 69. Accuracy

ValueTable.Name	Accuracy
ValueTable.Value	Integer
Description	This is the accuracy in meters for current user location
Range	-

FreeTextLocation

This is the location of the publisher as stated by the publisher himself. The content is a short text string.

Defined information elements are:

Table 70. FreeTextLocation

AttributeName.Name	FreeTextLocation
ValueTable.Name	PresenceValue
ValueTable.Value	Free text format
Description	This is the free text form of user location
Range	-

PLMN

This gives the PLMN name or code of the mobile network where the client device is currently registered.

Defined information elements are:

Table 71. PLMN

AttributeName.Name	PLMN
ValueTable.Name	PresenceValue
ValueTable.Value	Free text
Description	A PLMN code or name
Range	-

Communication capabilities

The client communication capabilities in the context of presence means the capability of the device hosting the client for various types of user-to-user communication.

Defined information elements are:

Table 72. CommCap

AttributeName	CommCap
ValueTable.Name	CommC
ValueTable.ValueTable	See [Table 73].
Description	The registration status of the client

Table 73. CommC

Name	CommC	Req	Description
ValueTable.Name	Cap	M	Communication capability of the communication client
ValueTable.Name	Status	M	Status of the communication client
ValueTable.Name	Contact	O	Contact address of the communication client
ValueTable.Name	Note	O	Free text describing the communication client

Table 74. Cap

ValueTable.Name	Cap
ValueTable.Value	CALL – a phone client SMS – an SMS client MMS – an MMS client IM – an IM client EMAIL – an e-mail client
Description	The communication clients represented with a predefined value.
Range	CALL, SMS, MMS, IM, EMAIL

Table 75. Status

ValueTable.Name	Status
ValueTable.Value	OPEN – the communication client can establish a connection with a remote client or a server CLOSED – the communication client cannot establish a connection with a remote client or server
Description	The current status of a communication client represented with a predefined value.
Range	OPEN, CLOSED

Table 76. Contact

ValueTable.Name	Contact
ValueTable.Value	1. When phone number see E.163 2. When mobile number see E.164 3. When MMS address see the WAP specifications 4. When UserID: see User-ID in the data types document 5. When email address: see RFC822
Description	Contact address depends on the value of Cap in the following way: Cap=Call – in this case Contact is a phone number Cap= SMS – in this case Contact is a mobile number (see E.164) Cap=MMS – in this case Contact is an MMS address Cap=IM – in this case the Contact is a UserID Cap=EMAIL – in this case the Contact is an e-mail address
Range	As defined in the appropriate reference: see Format.

Table 77. Note

ValueTable.Name	Note
ValueTable.Value	Free text format
Description	This is a user readable description of the communication client
Range	Max. 40 characters.

UserAvailability

This indicates the current status of the publisher in terms of amount of distraction he is willing to accept. The attribute indicates the availability of the publisher for telephony or messaging.

Defined information elements are:

Table 78. UserAvailability

AttributeName.Name	UserAvailability
ValueTable.Name	PresenceValue
ValueTable.Value	One of the following values: AVAILABLE – User is available for communication NOT_AVAILABLE – User is not immediately available for communication DISCREET – User selectively available depending on his personal status (e.g. busy)
Description	Defines the availability attribute
Range	AVAILABLE, NOT_AVAILABLE, DISCREET

PreferredContacts

This indicates the current preferred contact method for the publisher. The address of the contact is also available. All phone numbers shall be in international format.

Defined information elements are:

Table 79. PreferredContacts

AttributeName	PreferredContacts
ValueTable.Name	AddrPref
ValueTable.ValueTable	See [Table 80].
Description	List of preferences and addresses.

Table 80. AddrPref

Name	AddrPref	Req	Description
ValueTable.Name	PrefC	M	A communication preference
ValueTable.Name	Caddr	M	A contact address
ValueTable.Name	Cstatus	M	Status of the communication preference
ValueTable.Name	Cname	O	Human-readable name of a communication preference
ValueTable.Name	Cpriority	O	Priority of the communication preference

Table 81. PrefC

ValueTable.Name	PrefC
ValueTable.Value	CALL – the publisher prefers to be called SMS – the publisher prefers to get an SMS MMS – the publisher prefers to get an MMS IM – the publisher prefers to get an IM EMAIL – the publisher prefers to get EMAIL
Description	The user preference
Range	CALL, SMS, MMS, IM, EMAIL

Table 82. Caddr

ValueTable.Name	Caddr
ValueTable.Value	When phone number see [14]. When mobile number see [15]. When MMS address, see the WAP specifications. When UserID, see User-ID in the data types document [3]. When email address, see RFC822.
Description	Contact address depends on the value of PrefC in the following way: PrefC=Call – in this case Caddr is the phone number PrefC= SMS – in this case Caddr is the mobile number (see E.164) PrefC=MMS – in this case Caddr is the MMS address PrefC=IM – in this case the Caddr is the UserID PrefC=EMAIL – in this case the Caddr is the email address
Range	-

Table 83. Cstatus

ValueTable.Name	Cstatus
ValueTable.Value	OPEN – user is available to use the associated communication address CLOSED – user is not available to use the associated communication address
Description	The status of communication preference
Range	OPEN, CLOSED

Table 84. Cname

ValueTable.Name	Cname
ValueTable.Value	String (Free text format)
Description	User defined name of the communication preference
Range	-

Table 85. Cpriority

ValueTable.Name	Cpriority
ValueTable.Value	An integer in the range from 0 to 255.
Description	The priority the user may set for the communication preference. The smallest number indicates the highest preference. There may be more than one contact preference having the same priority value.
Range	0 to 255

PreferredLanguage

This indicates the preferred language for the publisher.

Defined information elements are:

Table 86. PreferredLanguage

AttributeName.Name	PreferredLanguage
ValueTable.Name	PresenceValue
ValueTable.Value	ISO 639-2/T (three letter language code)
Description	Language preference of the user
Range	-

StatusText

This is a short text string that gives a free form description of user status.

Defined information elements are:

Table 87. StatusText

AttributeName.Name	StatusText
ValueTable.Name	PresenceValue
ValueTable.Value	Free text format
Description	A personal status given as a free text
Range	-

StatusMood

This is the mood of the user. The personal mood of the user. It communicates the mood of a user to other users and is intended to be catalyst to further interactions between users, for example instant messages “Why are you angry?”

Defined information elements are:

Table 88. StatusMood

AttributeName.Name	StatusMood
ValueTable.Name	PresenceValue
ValueTable.Value	One of the following values: HAPPY – publisher feels happy SAD – publisher feels sad ANGRY – publisher feels angry JEALOUS – publisher feels jealous ASHAMED – publisher feels ashamed INVINCIBLE – publisher feels invincible IN_LOVE – publisher feels being in love SLEEPY – publisher feels sleepy BORED – publisher feels bored EXCITED – publisher feels excited ANXIOUS – publisher feels anxious
Description	A personal mood of the publisher given as an enumerated string
Range	HAPPY, SAD, ANGRY, JEALOUS, ASHAMED, INVINCIBLE, IN_LOVE, SLEEPY, BORED, EXCITED, ANXIOUS

Alias

This is a short text with the alias of the user

Defined information elements are:

Table 89. Alias

AttributeName.Name	Alias
ValueTable.Name	PresenceValue
ValueTable.Value	Free text format
Description	Alias name for the publisher
Range	-

StatusContent

Multimedia object content or URL to the multimedia content that the user has selected as personal status information.

Defined information elements are:

Table 90. StatusContent

Name	StatusContent	Req	Description
ValueTable.Name	DirectContent	C	A media object included into the attribute in transfer encoded form
ValueTable.Name	ReferredContent	C	An URL to the multimedia object
ValueTable.Name	ContentType	M	MIME type of the media object

Table 91. DirectContent

ValueTable.Name	DirectContent
ValueTable.Value	String
Description	A BASE64 encoded media object
Range	-

Table 92. ReferredContent

ValueTable.Name	ReferredContent
ValueTable.Value	String
Description	An URL to the multimedia content
Range	-

Table 93. ContentType

ValueTable.Name	ContentType
ValueTable.Value	MIME type indicator token
Description	MIME type of the media object
Range	-

ContactInfo

Contact information (vCard) or link to the contact information of the user.

Defined information elements are:

Table 94. ContactInfo

Name	ContactInfo	Req	Description
ValueTable.Name	ContainedvCard	C	A vCard included into the attribute in transfer encoded form
ValueTable.Name	ReferredvCard	C	An URL to the vCard

Table 95. ContainedvCard

ValueTable.Name	ContainedvCard
ValueTable.Value	Vcard
Description	A vCard as the ContactInfo attribute
Range	-

Table 96. ReferredvCard

ValueTable.Name	ReferredvCard
ValueTable.Value	URL
Description	An URL to resource having type vCard
Range	-

InfoLink

A set of URLs that the user has selected for extra information. The extra information can be of any content type.

Defined information elements are:

Table 97. InfoLink

AttributeName	InfoLink
ValueTable.Name	Inf_link
ValueTable.ValueTable	See [Table 98].
Description	One or more link elements.

Table 98. Inf_link

Name	Inf_link	Req	Description
ValueTable.Name	Link	M	URL to extra information.
ValueTable.Name	Text	O	A description of the link.
ValueTable.Name	ContentType	O	MIME type of the document referred to by Link.

Table 99. Link

ValueTable.Name	Link
ValueTable.Value	URL
Description	An URL to user's extra information.
Range	-

Table 100. Text

ValueTable.Name	Text
ValueTable.Value	Free text format
Description	A textual description of the link.
Range	-

Table 101. ContentType

ValueTable.Name	ContentType
ValueTable.Value	MIME type
Description	MIME type of the document referred to by the link.
Range	-

Appendix B: Extended presence information

This is a set of manufacturer, vendor or service provider specific extension attributes and extension fields. The extension attribute names are prefixed with 'Ext'. A client that does not understand the extension namespace shall ignore all the extension fields.

The stakeholder defining an extension attribute also defines the namespace for the attribute.

Table 102. Example of an extension attribute

AttributeName.Name	Ext:Logos
AttributeName.Namespace	http://www.nokia.com/PresenceLTGW1.0
ValueTable.Name	Name of the value
ValueTable.Value	Value
Description	Description of the attribute
Range	Possible value range

Appendix C: Examples of Presence WSI SOAP messages

Below are two examples of the SOAP requests/responses for GetPresence and UpdatePresence. These examples use Presence WSI 1.0.

GetPresence

==== Request ====

```
POST /axis/services/PSWSIPort HTTP/1.0
Content-Type: text/xml; charset=utf-8
Accept: application/soap+xml, application/dime,
multipart/related, text/*
User-Agent: Axis/1.1beta
Host: localhost
Cache-Control: no-cache
Pragma: no-cache
SOAPAction:
"http://www.nokia.com/schemas/Presence/v1.0/?operation=GetP
resence"
Content-Length: 588
Authorization: Basic cHNodHRwdXNlcjpwd2h0dHBwd2Q=
```

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <GetPresenceRequest
xmlns="http://www.nokia.com/schemas/Presence/v1.0/">
      <UserId>wsiappuser</UserId>
      <UserIdTable>
        <UserId>requesteduser</UserId>
      </UserIdTable>
      <AttributeNameTable>
        <AttributeName>
          <Name>OnlineStatus</Name>
        </AttributeName>
      </AttributeNameTable>
    </GetPresenceRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

==== Response ====

```
HTTP/1.1 200 OK
Pragma: No-cache
Cache-Control: no-cache
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Type: text/xml; charset=utf-8
Date: Tue, 25 Mar 2003 07:48:36 GMT
Server: Apache Coyote/1.0
Connection: close
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <GetPresenceResponse
xmlns="http://www.nokia.com/schemas/Presence/v1.0/">
      <Status>
        <StatusCode>1000</StatusCode>
        <StatusText>Operation was successful</StatusText>
      </Status>
      <AttributeAssociationTable>
        <AttributeAssociation>
          <UserId>requesteduser</UserId>
          <PresenceAttributeTable>
            <PresenceAttribute>
              <AttributeName>
                <Name>OnlineStatus</Name>
                <Namespace>http://www.wireless-
village.org/PA1.1</Namespace>
              </AttributeName>
              <UpdateTime>2003-03-25T07:48:28.671Z</UpdateTime>
              <Qualifier>true</Qualifier>
              <ValueTable>
                <NamedValue>
                  <Name>PresenceValue</Name>
                  <Value>F</Value>
                </NamedValue>
              </ValueTable>
            </PresenceAttribute>
          </PresenceAttributeTable>
        </AttributeAssociation>
      </AttributeAssociationTable>
    </GetPresenceResponse>
  </soapenv:Body>
</soapenv:Envelope>

```

UpdatePresence

==== Request ====

```

POST /axis/services/PSWSIPort HTTP/1.0
Content-Type: text/xml; charset=utf-8
Accept: application/soap+xml, application/dime,
multipart/related, text/*
User-Agent: Axis/1.1beta
Host: localhost
Cache-Control: no-cache
Pragma: no-cache
SOAPAction:
"http://www.nokia.com/schemas/Presence/v1.0/?operation=Upda
tePresence"
Content-Length: 768
Authorization: Basic cHNodHRwdXNlcjpwd2h0dHBwd2Q=

```

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <UpdatePresenceRequest
xmlns="http://www.nokia.com/schemas/Presence/v1.0/">
      <UserId>alice@nokia.com</UserId>
      <PresenceAttributeTable>
        <PresenceAttribute>
          <AttributeName>
            <Name>UserAvailability</Name>
          </AttributeName>
          <Qualifier>true</Qualifier>
          <ValueTable>
            <NamedValue>
              <Name>PresenceValue</Name>
              <Value>AVAILABLE</Value>
            </NamedValue>
          </ValueTable>
        </PresenceAttribute>
      </PresenceAttributeTable>
    </UpdatePresenceRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

```
==== Response ====
HTTP/1.1 200 OK
Pragma: No-cache
Cache-Control: no-cache
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Type: text/xml; charset=utf-8
Date: Tue, 25 Mar 2003 08:40:49 GMT
Server: Apache Coyote/1.0
Connection: close
```

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <PresenceResponse
xmlns="http://www.nokia.com/schemas/Presence/v1.0/">
      <Status>
        <StatusCode>1000</StatusCode>
        <StatusText>Operation was successful</StatusText>
      </Status>
    </PresenceResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

References

1. Wireless Village: Features and Functions, version 1.1:
http://www.openmobilealliance.org/wirelessvillage/docs/WV_Features_Functions_v1.1.pdf
2. Wireless Village: Presence Attributes, version 1.1:
http://www.openmobilealliance.org/wirelessvillage/docs/WV_PA_v1.1.pdf
3. Wireless Village: Client-Server Protocol Session and Transactions, version 1.1:
<http://www.openmobilealliance.org/wirelessvillage/>
4. W3C Note 08 May 2000 "Simple Object Access Protocol (SOAP) 1.1", URL:
<http://www.w3.org/TR/SOAP>
5. W3C Note Web Services Definition Language, WSDL 1.1 specification:
<http://www.w3.org/TR/wsdl>
6. Apache Axis, Apache's SOAP implementation:
<http://xml.apache.org/axis/>
7. Java™ 2 Platform, Standard Edition (J2SE™):
<http://java.sun.com/j2se/>
8. Apache Tomcat, Tomcat is the servlet container that is used in the official Reference Implementation for the Java Servlet and JavaServer Pages technologies:
<http://jakarta.apache.org/tomcat/>
9. IETF; RFC 2616: "Hypertext Transfer Protocol, HTTP/1.1", URL:
<http://www.ietf.org/rfc/rfc2616.txt>
10. IETF; RFC 2617: "HTTP Authentication: Basic and Digest ", URL:
<http://www.ietf.org/rfc/rfc2617.txt>
11. IETF; RFC 2396: "Uniform Resource Identifiers (URI): Generic Syntax", URL:
<http://www.ietf.org/rfc/rfc2396.txt>
12. W3C Note 15 September 1997 "Date and Time Formats" ISO 8601:
<http://www.w3.org/TR/NOTE-datetime>
13. Location Inter-operability Forum (LIF): Mobile Location Protocol, Version 2.0.0 20 Nov 2001
14. E.163: ITU-T Recommendation E.163 (11/88), Numbering Plan for the International Telephone Service
15. E.164: ITU-T Recommendation E.164 (05/97) The international public telecommunication numbering plan
16. OMA IMPS version 1.2 specifications:
http://www.openmobilealliance.org/agreement/LicenseAgreement.asp?DocName=OMA-IMPS-v1_2-20030221-C.zip

Glossary

Attribute list	A list of presence attributes that defines the attributes that are authorised for a particular subscriber or contact list.
Client-Server-Protocol (CSP)	A protocol between networked computers in which the services of one computer (the server) are requested by the other (the client). The client-server protocol can use different bearers depending on the capacity of the client. In Wireless Village or OMA IMPS, the client is a mobile terminal and the server is a presence server. For more information on CSP, refer to [3] and [16].
Contact List	A list of users in Nokia Presence Server. The contact list is used as a distribution list when subscribing presence and as a mechanism for proactive presence authorisation.
HTTP	Hypertext Transfer Protocol
ID	Identifier
IMPS	Instant Messaging and Presence Service. A service enabling the exchange of text and multimedia content between clients, as well as the exchange of presence information between networks and clients.
MSISDN	Mobile subscriber international ISDN number
OMA	Open Mobile Alliance. An organisation that acts as a mobile industry standards forum aiming at interoperable mobile services across geographic areas, network operators, and mobile terminals and at an open standards-based framework that permits services in a multi-vendor environment.
OMA IMPS	Open Mobile Alliance Instant Messaging and Presence Service. Formerly known as Wireless Village. For more information on OMA IMPS, refer to <i>OMA IMPS version 1.2 specifications [16]</i> .
PLMN	Public Land Mobile Network. A mobile network for the specific purpose of providing land mobile communication services to the public.
Presence Attribute (PA)	A piece of presence information intended for the user or meta-information for machine-to-machine communication between the publishing client and receiving clients. A client originated presence attribute is one which has its value field filled in by the publishing client. A server originated presence attribute is one which has its value field filled in by the publisher server.
Presence information	A collection of presence attributes which the presence server gathers and distributes. Presence information includes for example the user's availability, location, communication preferences, or any information that is valuable to subscriber and that the publisher wants to share. The end user has full control over the information that he is sharing with other users. Presence information is presented in the presence phonebook or contacts application of the subscriber's mobile device. Presence information is updated dynamically to subscribing terminals when changes occur.

Presence publisher	User of the presence service who is allowed to publish and update his presence status, and control the availability of presence information to others. The publisher may switch off the link between changes in the mobile phone profile and availability status.
Presence Server (PS)	Server software product that stores and distributes presence information between mobile terminals and enables the usage of presence enhanced phonebook application in terminals. Nokia Presence Server provides a scalable presence information storage and distribution engine for telecommunications operators.
SOAP	Simple Object Access Protocol
WAP	Wireless application protocol
Watcher list	A system-defined list of Nokia Presence Server users that have subscribed to the publisher's presence information. Nokia Presence Server maintains one watcher list for each user. It includes every user that has subscribed to one or more presence attributes, whether or not they were actually granted the access to the attributes to which they subscribed.
Wireless Village	An open industry initiative to create an open, common protocol for providing interoperable mobile instant messaging and presence services among workstations, network application servers, and mobile information appliances, such as mobile stations, handheld computers, and personal digital assistants (PDA). Now called OMA IMPS.
WSDL	Web Service Definition Language
WSI	Web Services Interface. OMA compliant web services interface through which businesses offer web services.
WV	see Wireless Village
XML	Extensible Markup Language

Index

A

Authentication, 39
authorisation, 39

C

Contact List, 14, 15, 16
 Attribute List, 14

D

Data types, 19

E

Error codes, 21

P

Presence Web Services Interface 1.0, 12, 17
Presence Web Services Interface 1.1, 40

U

Usage scenario

Contact lists and Authorisation, 14
Login and Presence update, 13
Presence query, 15
Presence subscription, 15
User search, 16

W

WSI operations, 23, 46
 CreateAttributeList, 33, 57
 CreateContactList, 29, 52
 DeleteAttributeList, 34, 58
 DeleteContactList, 30, 54
 GetAttributeList, 35, 58
 GetContactLists, 31, 54
 GetPresence, 23, 46
 GetSubscriptionList, 27, 50
 GetWatcherList, 28, 51
 ManageContactList, 32, 55
 PresenceNotification, 27, 50
 Search, 36, 60
 StopSearch, 38, 62
 SubscribePresence, 25, 48
 UnsubscribePresence, 26, 49
 UpdatePresence, 24, 47